

Master Thesis

Design and Evaluation of Efficient Semantic Company Retrieval at Scale

Marmee Pandya
(matriculation number 1963521)

July 29, 2026

Submitted to
Chair of Enterprise Data Analysis
Dr. Ralph Peeters
University of Mannheim

Abstract

Company search platforms that rely on manually configured keyword filters suffer from the vocabulary mismatch problem: a company that is a strong match for a search intent may never be surfaced if its own description does not repeat the query’s exact wording. This thesis investigates whether embedding-based semantic retrieval, combined with approximate nearest-neighbour search, can address this gap for ISTARI.AI’s company search product, evaluated throughout against Istari’s own production system rather than a synthetic benchmark. Over a corpus of 98,716 companies drawn from Istari’s 20-million-company database, more than ten dense embedding models, a late-interaction retrieval architecture, and several hybrid retrieval pipelines are compared under a shared evaluation protocol. A learned fusion ranker trained directly on the task’s own queries reaches Recall@1000 of 0.823, a 0.019 improvement over the best pipeline built purely from off-the-shelf pretrained components. Because no human relevance judgements exist for this task, an independent, multi-judge gold standard is constructed and used to re-check the fusion ranker’s own interpretive findings, revealing that production’s search results miss 27.3% of independently verified relevant companies outright, and that one previously dominant retrieval signal was in fact an artefact of the circular evaluation protocol it was first measured under. The pipeline is then scaled to a nested corpus of up to 397,025 companies, where approximate nearest-neighbour search matches exact search’s retrieval quality within 2.5% while running up to 78 times faster, confirming that the accuracy-efficiency trade-off central to this thesis’s research question remains favourable well beyond the original evaluation corpus.

Contents

Abstract	ii
1 Introduction	1
2 ISTARI.AI and its Data	3
2.1 About ISTARI.AI	3
2.2 The Global Organisation Index (GOI)	3
2.3 Data Pipeline and Methodology	3
2.3.1 Schema and Key Fields	4
2.3.2 Industry Classification: NACE Codes	5
2.3.3 Organisation Size Classification	5
2.3.4 Geographic Coverage	6
2.3.5 Evaluation Dataset	6
3 Literature	7
3.1 Sparse and Dense Retrieval	7
3.2 Approximate Nearest Neighbour Search	8
3.3 Late Interaction and Retrieval-Specific Pretraining	8
3.4 Hybrid Retrieval, Reranking, and Learning to Rank	8
3.5 Evaluation Without Human Relevance Judgements	9
4 Experiments	10
4.1 Experimental Setup	10
4.1.1 Dataset	10
4.1.2 Evaluation Protocol	11
4.1.3 Computational Environment	11
4.2 Sparse Retrieval Baseline: BM25	12
4.2.1 Implementation	12
4.2.2 Qualitative Analysis	12
4.3 Dense Retrieval Baselines	12
4.3.1 all-MiniLM-L6-v2 (MiniLM)	13
4.3.2 BGE-large-en-v1.5 (BGE)	13
4.3.3 OpenAI text-embedding-3-small (OpenAI)	13
4.4 Results	14
4.4.1 Retrieval Quality	14
4.5 Runtime Analysis	15
4.6 Evaluation Protocol Change for Follow-Up Experiments	17
4.7 Extended Baseline Comparison: Additional Embedding Models	17

Contents

4.8	BGE Instruction-Prefix Correction	19
4.9	Query Expansion: HyDE Pipeline	20
4.10	Hybrid Retrieval: Reciprocal Rank Fusion	21
4.11	LLM-as-Judge Relevance Validation	21
4.12	Production API Evaluation and MiniLM Rerank	22
4.13	ANN Index Scaling to 2 Million Vectors	23
4.14	Recall@1000 Error Analysis	23
4.15	Corpus Quality: Platform Boilerplate in the AI-Generated Summary . . .	24
4.16	Oracle Union Recall Ceiling	25
4.17	SimLM: A Purpose-Built Retrieval Pre-training Baseline	26
4.18	ColBERT: Late-Interaction Retrieval from Scratch	27
4.19	Hybrid Full-Pool Reranking	28
4.20	Reranker Ablation: Isolating the Bottleneck	29
4.21	Learned Fusion Ranker	30
4.22	Validating the Learned Fusion Ranker Against a Non-Circular Gold Stan- dard	31
4.22.1	Building a Pooled, Independently Judged Gold Standard	31
4.22.2	Retraining Against the New Gold Standard	32
4.23	Scaling Beyond the Gold Standard: Silver Labels via Model Distillation .	34
4.23.1	Expanding the Gold Standard via Active Learning	35
4.23.2	Deepening a Headline Query Subset via Round-Based Active Learn- ing	37
4.23.3	Quantifying How Far Production’s Own Ranking Diverges from the Independent Gold/Silver Set	39
4.23.4	A Direct Ranking Comparison: Production vs. an Independently Cross-Validated Ranker	40
4.23.5	Is the Classifier’s Confidence Honest? A Calibration Check	41
4.23.6	Honest Synthesis: What Has This Thesis’s Validation and Exper- imentation Actually Shown?	42
4.23.7	Preparing an External Review: Where Existing Gold Coverage Actually Sits in the Ranking	45
4.23.8	Hyperparameter Tuning the Fusion Ranker	46
4.23.9	A Neural Network Fusion Ranker	48
4.23.10	Fine-Tuning the Reranker	49
4.24	Scaling to a Larger Corpus: a Random Sample and Nested Corpus Tiers .	51
4.24.1	Approximate Nearest Neighbor (ANN) Index Tuning	54
5	Conclusion and Future Work	57
5.1	Summary of Contributions	57
5.2	Future Work	58
	Bibliography	60
	Ehrenwörtliche Erklärung	63

1 Introduction

Modern company intelligence platforms maintain datasets containing tens of millions of organisation profiles, covering everything from small local businesses to large multinational enterprises. Despite the availability of this data, finding the companies that are actually relevant to a given search intent remains difficult in practice. Existing workflows typically rely on manually configured keyword filters, which require the user to guess the exact vocabulary a relevant company happens to use in its own description, and which need multiple rounds of trial and error before returning a satisfactory set of results. This is a version of a well-known limitation of classical keyword-based retrieval: two texts can describe the same underlying concept using entirely different words, so a company that is a strong match for a query may never be surfaced simply because it does not happen to repeat the query’s own terminology.

At the same time, any system intended to operate on a company database of this size must satisfy strict constraints on efficiency and scalability. Retrieval has to remain accurate while operating over a large-scale, high-cardinality dataset, and it has to return results quickly enough to support an interactive search experience, not just a slow, offline batch process.

This thesis investigates whether embedding-based semantic retrieval, where both queries and companies are represented as dense vectors and matched by meaning rather than by literal word overlap, can address this gap. Concretely, the thesis asks: can embedding-based semantic retrieval, combined with approximate nearest-neighbour search, efficiently generate accurate top- k candidate company sets from a large-scale company dataset, while maintaining low query latency, and how does it compare to the keyword-based production system currently in use?

This work is conducted in collaboration with ISTARI.AI, a data intelligence company whose core product is a curated, organisation-level dataset covering companies world-wide. The collaboration grounds the thesis in a real operational setting: the queries evaluated throughout this thesis are real search intents drawn from Istari’s own usage, and the production system this thesis compares against is the system Istari’s users interact with today, rather than a synthetic or purely academic benchmark.

Contributions. This thesis makes the following contributions. First, it presents a systematic, large-scale comparison of retrieval approaches for company search, spanning classical lexical retrieval, more than ten dense embedding models of varying size and training objective, a late-interaction retrieval architecture, and a retrieval-specific pre-trained encoder, all evaluated on the same corpus, query set, and metrics, so that results are directly comparable across methods. Second, it develops and evaluates a series of hybrid retrieval pipelines, culminating in a learned fusion ranker that is trained directly

1 Introduction

on this task’s own queries and outperforms every pipeline built purely from frozen, pretrained components, while also yielding an interpretable account of which retrieval signals actually drive relevance. Third, it evaluates the accuracy-efficiency trade-off of approximate nearest-neighbour search at simulated production scale, and provides a decomposed latency analysis that separates local computation from network overhead for API-based methods. Fourth, it identifies, quantifies, and discusses a previously unquantified data-quality issue in the underlying corpus, in which a meaningful share of AI-generated company summaries describe a website hosting or platform provider rather than the company itself, and shows both how much this affects retrieval and why it was not corrected in the results reported here. Finally, because no human-annotated relevance judgements exist for this task, the thesis is explicit throughout about the limitations of the pseudo-relevance evaluation protocol it adopts instead, and includes an independent validation of that protocol against large language model-based relevance judgements.

Thesis structure. Chapter 2 introduces Istari’s Global Organisation Index, the dataset underlying every experiment in this thesis, describing how it is constructed, its schema, and the evaluation corpus derived from it. Chapter 3 reviews the retrieval paradigms, indexing techniques, and evaluation practices that this thesis builds on and compares against, including sparse and dense retrieval, approximate nearest-neighbour search, late-interaction retrieval, hybrid retrieval and learning to rank, and evaluation practice in the absence of human relevance judgements. Chapter 4 presents the experimental work itself: the baseline retrieval systems and evaluation protocol, an extended comparison across many additional embedding models, several hybrid and query-expansion strategies, an evaluation-methodology check against large language model-based judgements, a comparison against Istari’s own production search APIs, an approximate nearest-neighbour scaling experiment, a detailed error analysis, the corpus data-quality finding concerning AI-generated summaries, and finally the learned fusion ranker that achieves the best result obtained anywhere in this thesis.

2 ISTARI.AI and its Data

2.1 About ISTARI.AI

This thesis is conducted in collaboration with ISTARI.AI¹, a data intelligence company that specialises in providing high-quality firmographic data at scale. Istari’s core product is a curated, organisation-level dataset that covers organisations worldwide, enabling use cases such as market analysis, company search, and lead generation. The central research challenge addressed in this thesis is improving the quality of semantic company retrieval, arises directly from Istari’s operational needs.

2.2 The Global Organisation Index (GOI)

The primary dataset used in this thesis is Istari’s **Global Organisation Index** (GOI), a curated collection of approximately 20 million active organisations spanning 232 countries and territories. The GOI serves as the canonical source of firmographic data within Istari’s data infrastructure, covering organisation identity, location, industry classification, size, etc.

2.3 Data Pipeline and Methodology

The GOI is constructed through a multi-stage validation pipeline that deliberately prioritises data quality over volume. The pipeline proceeds as follows:

1. **Registry Collection.** National registers are systematically queried worldwide, including commercial registers, company registers, and association registers. These primary sources are enriched with additional open data sources, yielding an initial pool of approximately 400 million organisations.
2. **Domain Attribution.** Each organisation is assessed for attribution to a clearly identifiable web domain. Approximately 10% of the initial pool, around 40 million organisations can be matched to a domain. The remainder are excluded as they are either no longer active, were never truly operational (e.g., pure holding structures), or lack a web presence.
3. **Activity Verification.** The 40 million domain-attributed organisations are further filtered by verifying that their domains are still actively operated. This final

¹https://docs.istari.ai/goi/goi_start_page/

step reduces the dataset to approximately 20 million organisations that are demonstrably active.

This pipeline ensures that every record in the GOI is both web domain-verified and confirmed active, a key differentiator from traditional firmographic databases, which typically retain large numbers of inactive or dormant records.

2.3.1 Schema and Key Fields

Each record in the GOI contains a set of core identity fields, geographic fields, and classification fields. The fields most relevant to this thesis are summarised in Table 2.1.

Table 2.1: Key fields of the GOI schema used in this thesis.

Field	Type	Description
name	STRING	Organisation name
domain	STRING	Organisation website domain (unique identifier)
summary	STRING	AI-generated textual summary of the organisation
keywords	LIST	Keywords
employee_class	STRING	Employee count bracket
country	STRING	Country
country_code	STRING	ISO Country Code
state / state_code	STRING	State or province
region / region_code	STRING	Region
district / district_code	STRING	District
municipality / municipality_code	STRING	Municipality
address	STRING	Full address
latitude	STRING	Latitude coordinate
longitude	STRING	Longitude coordinate
nace_code	STRING	NACE industry classification code
organization_type	STRING	Company, Startup, Academic, Public, or Other
organization_size	STRING	Micro, Small, Medium-sized, or Large enterprise
country	STRING	Country of registration

Of particular importance for this thesis is the **summary** field. This field contains an AI-generated natural language description of each organisation, synthesised from its web presence and available structured data. It is the most semantically rich and consistently populated field in the dataset, with effectively zero missing values across the corpus used for experiments, and therefore serves as the primary input for all retrieval models evaluated in this thesis.

2.3.2 Industry Classification: NACE Codes

Industries in the GOI are classified using the **NACE** taxonomy² (*Nomenclature statistique des activités économiques dans la Communauté européenne*), the European Union’s standard statistical classification of economic activities. The GOI covers 22 NACE sectors. Table 2.2 lists the ten most represented sectors in the dataset.

Table 2.2: Top 10 industries in the GOI by organisation count.

NACE	Industry	Count	Share (%)
N	Professional, scientific & technical activities	3,093,879	18.03
G	Wholesale & retail trade	2,240,191	13.05
C	Manufacturing	2,006,435	11.69
R	Human health & social work	1,440,340	8.39
K	Telecom, IT & computing	1,269,131	7.39
F	Construction	1,173,226	6.84
I	Accommodation & food service	1,126,594	6.56
S	Arts, sports & recreation	1,101,592	6.42
Q	Education	584,430	3.41
J	Publishing, broadcasting & content	554,985	3.23

2.3.3 Organisation Size Classification

Organisation size in the GOI is derived from employee and revenue signals and bucketed into four categories following the European Union’s definitions of small and medium-sized enterprises (SME), as shown in Table 2.3.

Table 2.3: Organisation size distribution in the GOI.

Size	Employee Range	Count	Share (%)
Micro	0–9	7,341,838	42.78
Small	10–49	7,494,409	43.67
Medium-sized	50–249	1,501,655	8.75
Large enterprise	250+	824,126	4.80

The distribution is heavily skewed towards smaller organisations: over 86% of the GOI consists of micro and small enterprises. This has implications for retrieval, as queries targeting large enterprises represent a smaller and more distinctive subset of the corpus.

²<https://ec.europa.eu/eurostat/web/nace>

2.3.4 Geographic Coverage

The GOI covers 232 countries and territories. Table 2.4 lists the top five countries by volume.

Table 2.4: Top 5 countries by organisation count in the GOI.

Country	Organisations
United States	3,626,794
Germany	1,828,181
United Kingdom	1,013,026
Netherlands	778,416
Italy	625,754

It is worth noting that approximately 2.7 million organisations in the GOI do not have a standardised administrative region. These records are intentionally retained as they remain valuable for non-geographic analyses such as industry or domain-level retrieval, the primary focus of this thesis.

2.3.5 Evaluation Dataset

The full 20-million-company GOI corpus is not directly used for retrieval in this thesis. Instead, Istari provided an evaluation dataset consisting of 101 real-world search queries along with each query’s top-1 000 production search results, yielding 101 000 (query, company) pairs in total. Where the same domain appears linked to multiple addresses, the GOI deduplicates by domain and retains the record with the highest employee count, treating it as the organisation’s headquarters; after this deduplication, the evaluation corpus covers **98 716 unique companies**. All retrieval experiments in Chapter 4 are conducted over this corpus. The exact pseudo-relevance threshold used to define “relevant” companies for evaluation (and how it changes across experiments) is described in Sections 4.1.2 and 4.6.

Testing retrieval quality at the full 20-million-company scale was not pursued in the current phase of this thesis, since doing so would require a substantially larger data export from Istari than the query-level top-1 000 results used here; this is discussed as an open question in the concluding chapter.

3 Literature

This chapter reviews the retrieval paradigms, indexing techniques, and evaluation practices that this thesis builds on and compares against. Section 3.1 covers sparse and dense retrieval, the two foundational paradigms for the baselines in Chapter 4. Section 3.2 covers approximate nearest neighbour search, the technique that makes dense retrieval practical at scale. Section 3.3 covers late-interaction retrieval and retrieval-specific pre-training, the two alternative paradigms evaluated in Sections 4.17–4.18. Section 3.4 covers hybrid retrieval, cross-encoder reranking, and learning to rank, the techniques underlying the hybrid pipelines in Sections 4.19–4.21. Section 3.5 covers evaluation practice in information retrieval when human relevance judgements are unavailable, directly motivating the pseudo-relevance protocol used throughout this thesis (Section 4.1.2).

3.1 Sparse and Dense Retrieval

Text retrieval systems fall into two broad families. **Sparse** retrieval represents queries and documents as high-dimensional vectors of term weights and scores matches by term overlap. The dominant sparse method is BM25 (Robertson and Zaragoza 2009), a probabilistic scoring function that weighs each query term by its frequency in a document and its rarity across the whole collection. BM25 is fast, requires no training, and remains a strong baseline decades after its introduction, but it cannot match a query and a document that describe the same concept using different words: this is the vocabulary mismatch problem, directly observed for this thesis’s task in Section 4.2.2.

Dense retrieval instead represents queries and documents as fixed-size vectors learned by a neural encoder, and scores matches by vector similarity rather than literal term overlap. Dense Passage Retrieval (DPR) (Karpukhin et al. 2020) established the now-standard bi-encoder architecture for open-domain retrieval: a query encoder and a document (or, in this thesis, company) encoder, trained so that relevant pairs are close in vector space, encoding each side independently so that documents can be embedded once, offline, and reused across every future query. Sentence-BERT (Reimers and Gurevych 2019) showed that the same bi-encoder idea, applied to general-purpose sentence embeddings rather than a retrieval-specific training objective, produces embeddings useful for semantic similarity and retrieval tasks broadly, and underlies the general-purpose sentence-transformer models (including MiniLM, Section 4.3.1) used throughout this thesis. More recent embedding models, such as the BGE family (Xiao et al. 2024; Chen et al. 2024), combine this bi-encoder architecture with large-scale contrastive pretraining and hard-negative mining, and represent the current state of the art for general-purpose dense retrieval, forming the basis of several baselines evaluated in Sections 4.3.2 and 4.7.

3.2 Approximate Nearest Neighbour Search

Dense retrieval requires finding the nearest vectors to a query vector among the full document collection. Exact search costs time linear in the number of documents, which becomes impractical at the scale of millions of documents relevant to this thesis’s target deployment (Section 2). Approximate nearest neighbour (ANN) search trades a small amount of accuracy for a large reduction in query time by searching only a carefully chosen subset of the collection instead of comparing the query against every document. FAISS (Johnson et al. 2019) provides GPU-accelerated implementations of both exact and approximate search, including inverted-file indexes (IVF), which cluster the embedding space and search only the clusters nearest the query, trading a small amount of recall for a large reduction in query time. Hierarchical Navigable Small World graphs (HNSW) (Malkov and Yashunin 2020) take a different approach: they build a multi-layer proximity graph over the document vectors and greedily traverse it at query time, again trading exactness for speed. Both IVF and HNSW are evaluated directly in this thesis’s ANN scaling experiment (Section 4.13), which tests whether the accuracy–speed trade-off these methods make remains favourable at simulated production scale.

3.3 Late Interaction and Retrieval-Specific Pretraining

A bi-encoder compresses an entire document into one fixed-size vector, which can discard fine-grained information relevant to a specific query. ColBERT (Khattab and Zaharia 2020) addresses this with *late interaction*: query and document are each represented as a set of per-token vectors, and relevance is scored by summing, for every query token, the maximum similarity to any document token (MaxSim), so no single vector has to summarise the whole document in advance. This adds compute and storage cost relative to a single-vector bi-encoder, since retrieval now requires comparing token sets rather than single vectors. ColBERTv2 (Santhanam et al. 2022) introduces residual compression of the per-token vectors and the PLAID engine for efficient late-interaction search at scale, substantially reducing this cost; this thesis instead implements MaxSim scoring directly (Section 4.18), trading the PLAID engine’s efficiency for an exact, dependency-light implementation suited to the scale of the evaluation corpus used here.

A separate line of work improves dense retrieval not through architecture but through pretraining. SimLM (Wang et al. 2022) pretrains a bi-encoder with a representation-bottleneck objective before any retrieval-specific fine-tuning, explicitly designed to produce embeddings well suited to downstream passage retrieval. This thesis evaluates whether that retrieval-specific pretraining transfers to a domain (company profiles) quite different from the passage-retrieval benchmarks it was designed for (Section 4.17).

3.4 Hybrid Retrieval, Reranking, and Learning to Rank

Sparse and dense retrieval, and different dense models, make different errors: a hybrid system that combines multiple retrieval signals can recover from a single method’s in-

dividual weaknesses. Reciprocal Rank Fusion (RRF) (Cormack et al. 2009) is a simple, training-free way to combine several ranked lists: each document’s fused score is the sum of $1/(k + \text{rank})$ across the lists it appears in, requiring only that a document be ranked, not that its raw scores be comparable across methods. RRF underlies the hybrid baseline in Section 4.10.

A complementary technique is *reranking*: a first-stage retriever returns a candidate set cheaply, and a more expensive model reorders it. Cross-encoder rerankers, such as the `bge-reranker-v2-m3` (BAAI 2024) and `mxbai-rerank-large-v1` (Mixedbread AI 2024) models used in Sections 4.19 and 4.20, feed the query and each candidate into the same transformer jointly, rather than encoding them independently as a bi-encoder does. This lets the model attend directly between query and candidate tokens, typically producing more accurate relevance scores than a bi-encoder, at the cost of requiring a full forward pass per candidate rather than a single fast vector comparison, so cross-encoders are used to rerank a small candidate set rather than to search a full collection.

More generally, combining multiple retrieval signals into a single relevance score can be framed as *learning to rank* (Liu 2009): instead of a fixed, hand-designed combination rule such as RRF, a model is trained directly on labelled query-document pairs to learn how to weight and combine whatever features are available (similarity scores, ranks, or other signals) into a final ranking. This is the approach taken by the learned fusion ranker in Section 4.21, implemented with the logistic regression and gradient-boosted tree models provided by scikit-learn (Pedregosa et al. 2011).

3.5 Evaluation Without Human Relevance Judgements

Standard information retrieval evaluation compares each system’s ranking against a set of human relevance judgements for a fixed query set, historically constructed through pooling: the top results from several systems are merged and judged by human assessors, as established by the methodology of the Text REtrieval Conference (TREC) (Voorhees 1999). No such human-annotated relevance judgements exist for the company search task addressed in this thesis. In their absence, this thesis instead adopts a *pseudo-relevance* protocol (Section 4.1.2): a company is treated as relevant if it appears in the existing production system’s own results for that query. This is a common practice when human judgements are unavailable, and the BEIR benchmark (Thakur et al. 2021) similarly demonstrates that retrieval methods can be compared consistently across many tasks and domains without task-specific human annotation, provided the resulting scores are interpreted as agreement with a reference system rather than as absolute measures of relevance, a limitation this thesis states explicitly wherever pseudo-relevance labels are used.

4 Experiments

This chapter describes the experimental setup, baseline retrieval systems, evaluation methodology, and results obtained for the company search task over Istari’s Global Organisation Index (GOI). Section 4.1 introduces the dataset and evaluation protocol. Sections 4.2–4.3 describe each retrieval baseline in turn. Section 4.4 presents and analyses the results across all methods, and Section 4.5 provides a runtime comparison. Section 4.6 documents a change in evaluation protocol adopted for all subsequent experiments. Sections 4.7–4.16 extend the baseline comparison, explore query expansion and hybrid fusion, validate the pseudo-relevance labels against a large language model (LLM) judge, compare against Istari’s production application programming interfaces (APIs), evaluate approximate nearest neighbour search at scale, analyse the sources of retrieval error, and document a corpus-level data-quality limitation affecting the AI-generated company summaries used throughout this thesis. Sections 4.17–4.18 evaluate two further retrieval paradigms (a retrieval-pretrained encoder and late-interaction retrieval). Sections 4.19–4.21 develop a full-pool reranking pipeline, isolate the reranker as a bottleneck, and close part of the remaining gap with a learned fusion ranker trained on this task’s own data, the best-performing method in this thesis.

4.1 Experimental Setup

4.1.1 Dataset

The evaluation corpus is derived from Istari’s production search logs. A set of 101 representative queries was provided by Istari, covering a wide range of company search intents, including industry-specific queries (e.g. “*pharmaceutical companies in the US*”), geographic queries (e.g. “*tech startups in Berlin*”), and functional queries (e.g. “*B2B SaaS companies*”). For each query, the production system returned its top-1 000 ranked results, yielding 101 000 query–company pairs in total. After de-duplication across queries, the corpus consists of **98 716 unique companies**, each represented by a domain name, a company name, and an AI-generated natural-language summary of the company’s business activity.

The summary field was selected as the primary retrieval field after an exploratory data analysis of all available GOI attributes. It was found to be fully populated across the corpus (0% missing values) and to contain the richest semantic content for matching against free-text queries, compared to structured fields such as industry codes or founding year.

4.1.2 Evaluation Protocol

Pseudo-relevance labels. No human-annotated relevance judgements are available for the Istari company search task. Following standard practice in information retrieval research when ground-truth labels are absent (Voorhees 1999), this thesis adopts a *pseudo-relevance* assumption: a company is considered **relevant** for a given query if and only if it appears in the **top-100 results** of Istari’s production search system for that query. This yields a relevance set of at most 100 companies per query, representing $\sim 0.1\%$ of the total corpus.

This assumption has an important implication for the evaluation: the metrics measure the degree to which each retrieval system agrees with the production system, not whether it recovers objectively relevant companies. Consequently, a retrieval system that is strictly better than the production system may appear to underperform because it surfaces companies the production system missed. This limitation is acknowledged throughout the analysis.

Metrics. Three standard information retrieval metrics are reported at cut-offs $k \in \{10, 50, 100, 1000\}$:

- **Precision@ k :** the fraction of the top- k retrieved companies that are relevant. This measures user-facing quality: what a user sees on the first page of results.
- **Recall@ k :** the fraction of all relevant companies that appear in the top- k results. This measures coverage, important for downstream re-ranking pipelines that operate on a retrieved candidate set. **Recall is the primary metric in this thesis**, following the priority set by Istari for this task: whether the system finds the relevant companies matters more than the exact order they are ranked in. Section 4.6 explains how Precision@ k and Recall@ k relate to one another and should be read together.
- **NDCG@ k** (Normalised Discounted Cumulative Gain): a ranking-aware metric that rewards systems for placing relevant companies higher in the list, using a logarithmic discount. NDCG@ k is reported as a secondary, supporting metric: it reflects the experience of a user who interacts primarily with the top results, which is a reasonable concern in practice even though it is not the priority metric for this thesis.

All metrics are averaged across all 101 queries. Precision and Recall are always reported at the same cut-off values so that the precision–recall trade-off at each depth is visible.

4.1.3 Computational Environment

All experiments are conducted on the University of Mannheim’s high-performance computing (HPC) cluster. Dense model encoding is performed on an NVIDIA A100 graphics

processing unit (GPU) with 80 GB HBM2 memory. BM25 retrieval runs on the central processing unit (CPU) instead, since it does not involve any neural network computation. Query latency measurements are performed on the same hardware and reported as the mean over 20 benchmark queries to obtain stable estimates. Latency is decomposed into three components (local query encoding, FAISS index search, and external network time) so that self-hosted and API-based methods are compared on equal terms.

4.2 Sparse Retrieval Baseline: BM25

4.2.1 Implementation

The BM25 baseline is implemented using the `rank_bm25` Python library (Robertson and Zaragoza 2009), which provides the Okapi BM25 variant with default hyperparameters $k_1 = 1.5$ and $b = 0.75$. All 98 716 company summaries are tokenised using the punkt tokeniser from the Natural Language Toolkit (NLTK) (Bird et al. 2009) with lower-casing applied, and the resulting token lists are used to construct the BM25 inverted index. At query time, each query is tokenised identically and scored against the index. The top-1 000 companies by BM25 score are retrieved for each query and stored for evaluation. Index construction takes 28.7 seconds and requires no GPU.

4.2.2 Qualitative Analysis

Manual inspection of BM25 results reveals a consistent failure mode arising from the *vocabulary mismatch problem*: BM25 retrieves companies that contain query keywords in their description, regardless of whether those companies operate in the queried domain.

A representative example is the query “*software companies*”. The top BM25 results are predominantly investment banks and venture capital firms whose summaries frequently mention the word *software* in the context of describing their investment portfolio (e.g. “*we invest in software, fintech, and SaaS businesses*”). These companies are not software companies: they fund them. BM25 cannot distinguish between a company *operating* in a domain and a company *discussing* that domain. This limitation provides a clear and concrete motivation for semantic retrieval methods that operate on the meaning of text rather than on token overlap.

4.3 Dense Retrieval Baselines

Three dense retrieval baselines are evaluated, covering a range of model sizes, computational requirements, and deployment constraints. All three follow the same two-stage pipeline introduced by Karpukhin et al. (2020): (i) encode all company summaries offline into a dense vector index using FAISS (Johnson et al. 2019), and (ii) at query time, encode the query into a vector and retrieve the top-1 000 nearest neighbours from the index.

4.3.1 all-MiniLM-L6-v2 (MiniLM)

`all-MiniLM-L6-v2` is a lightweight sentence-transformer model (Reimers and Gurevych 2019) with 22 million parameters, producing 384-dimensional embeddings. It is distilled from a larger BERT model and fine-tuned on sentence similarity tasks. All 98 716 company summaries are encoded in a single batch pass, taking 0.6 minutes on the A100 GPU. A `faiss.IndexFlatL2` index (exact Euclidean search) is constructed over the resulting embeddings in 0.2 seconds. MiniLM does not require embedding normalisation.

MiniLM is included as the smallest and most computationally efficient dense baseline, providing a lower bound on dense retrieval quality and establishing whether a minimal dense model is sufficient to outperform BM25.

4.3.2 BGE-large-en-v1.5 (BGE)

`BAAI/bge-large-en-v1.5` is a 335-million parameter BERT-large model developed by the Beijing Academy of Artificial Intelligence (Xiao et al. 2024), producing 1 024-dimensional embeddings. It is specifically optimised for dense retrieval through hard negative mining during fine-tuning, a training strategy in which the model is exposed to query–document pairs that are superficially similar but semantically distinct, forcing it to learn fine-grained semantic distinctions.

Encoding all 98 716 company summaries takes 7.0 minutes on the A100 GPU at a batch size of 256. BGE requires L2 normalisation of all embeddings (`normalize_embeddings=True`), as it is trained under the cosine similarity objective. Accordingly, a `faiss.IndexFlatIP` index (inner product, equivalent to cosine similarity for unit-normalised vectors) is used rather than `IndexFlatL2`.

At query time, each query is encoded independently and normalised before being passed to the FAISS index. The query encoding step takes 16.6 ms on the A100 GPU, and the subsequent FAISS search takes 27.8 ms, for a total query latency of 44.4 ms.

4.3.3 OpenAI text-embedding-3-small (OpenAI)

`text-embedding-3-small` is a commercial embedding model provided by OpenAI via their REST API. It produces 1 536-dimensional embeddings. Unlike the open-source models above, the architecture and training procedure are proprietary. Encoding all 98 716 company summaries is performed via batched API calls (batch size 500) and takes 9.7 minutes, of which the majority is network latency rather than model computation. The resulting embeddings are unit-normalised by the API, and a `faiss.IndexFlatIP` index is constructed locally. The total cost of encoding the full corpus is approximately \$0.15 at the published pricing of \$0.02 per million tokens.

This model is included as the commercial upper-bound baseline, following a suggestion from the industry supervisor that a high-quality API-based embedding model be evaluated alongside open-source alternatives.

Latency note. Query latency for the OpenAI baseline comprises two components: a network round-trip to the OpenAI API servers (211.7 ms, measured over 20 queries)

and a local FAISS search (42.9 ms). The network component is not a property of the model itself; it reflects internet latency between the University of Mannheim cluster and OpenAI’s infrastructure, and should not be interpreted as a measure of model speed. For a fair comparison, the local FAISS search time (42.9 ms) is comparable to BGE’s search time (27.8 ms), confirming that both methods perform equivalently at the index search stage. The substantive difference is that OpenAI requires an external API call for query encoding, whereas BGE encodes locally on GPU in 16.6 ms.

4.4 Results

4.4.1 Retrieval Quality

Recall is the primary metric reported in this thesis, following the priority set by Istari for this task: coverage of relevant companies matters more than precise ranking of the first page of results. Table 4.1 presents Recall@ k for all four methods; NDCG and Precision are discussed alongside it in the paragraphs below, where individual values are relevant, rather than tabulated in full.

Table 4.1: Recall@ k of all four baseline systems averaged over 101 queries. Bold indicates the best value in each column.

Method	Recall@10	Recall@50	Recall@100	Recall@1000
BM25	0.016	0.068	0.128	0.682
MiniLM	0.019	0.069	0.122	0.653
BGE	0.022	0.087	0.159	0.796
OpenAI	0.022	0.080	0.149	0.803

BM25 vs dense retrieval. All three dense methods outperform BM25 at low cut-offs. At $k = 10$, BGE and OpenAI achieve an NDCG of 0.227, representing a **37% improvement** over BM25’s 0.166. Precision@10 improves from 0.156 (BM25) to 0.216 (BGE), a 38% relative gain. This confirms the hypothesis that semantic retrieval better captures the intent of natural-language company search queries, particularly for cases where relevant companies use vocabulary different from the query terms.

At large cut-offs ($k = 1000$), the gap narrows. BM25 achieves a Recall@1000 of 0.682, compared to BGE’s 0.796 and OpenAI’s 0.803. This pattern is consistent with findings in the BEIR benchmark (Thakur et al. 2021): dense models are stronger at ranking precision, while keyword-based methods maintain competitive coverage at large retrieval depths due to their broad lexical matching.

BGE vs OpenAI. BGE and OpenAI are **effectively equivalent** across all metrics and cut-offs. The largest observed difference is 0.013 NDCG points (at $k = 50$), which is within the range of query-level variance and should not be interpreted as a meaningful

4 Experiments

quality advantage. This finding has an important practical implication: a free, self-hosted model (BGE) achieves the same retrieval quality as a commercial API (OpenAI) on this task.

MiniLM. MiniLM consistently underperforms BGE and OpenAI at all cut-offs, and is only marginally better than BM25 at large k (Recall@1000: 0.653 vs 0.682 for BM25). It is also slower than BGE in production (query latency 80.9 ms vs 44.4 ms), as discussed in Section 4.5. MiniLM is therefore dominated by BGE on both quality and speed, and there is no operating point at which it represents the optimal choice.

Precision and the low-prevalence setting. Absolute precision values are low across all methods (Precision@10 ranging from 0.156 to 0.224). This is expected: relevant companies represent at most 100 out of 98 716 corpus entries, approximately 0.1% of the corpus. In such a low-prevalence setting, even a perfect retrieval system would show low absolute precision at large k . What matters is the relative improvement between methods: BGE and OpenAI achieve 38–43% higher Precision@10 than BM25, which is a meaningful gain.

The NDCG dip at $k = 100$. An artifact of the evaluation design is visible in Table 4.1: NDCG values decrease from $k = 10$ to $k = 100$ before rising sharply at $k = 1000$ for all methods. This occurs because the pseudo-relevance labels are defined as the production top-100. At exactly $k = 100$, NDCG measures whether all 100 relevant companies are ranked perfectly within the top-100 retrieved results, a maximally strict condition. At $k = 1000$, there is substantially more room to accommodate the relevant set, and the metric rises accordingly. This artifact would not appear with human-annotated relevance labels that include varying degrees of relevance beyond a hard top-100 cut-off.

4.5 Runtime Analysis

Table 4.2 presents the decomposed query latency for all four methods. Latency is broken into three components to enable a fair comparison between self-hosted models and the API-based OpenAI baseline.

BGE is the fastest method overall. BGE achieves a total query latency of 44.4 ms, faster than BM25 (104.1 ms), MiniLM (80.9 ms), and OpenAI (254.6 ms). This result is initially counterintuitive: BGE is a 335-million parameter neural network, whereas BM25 is a simple keyword scoring function. The explanation lies in hardware utilisation. BGE encodes the query vector on the A100 GPU (16.6 ms), a highly parallelised operation that is extremely efficient for dense linear algebra. BM25, by contrast, computes token scores on CPU over an inverted index (104.0 ms), which is sequential and memory-bound. This finding has a direct implication for production deployment: a GPU-equipped system running BGE delivers both better retrieval quality and lower query latency than a CPU-based BM25 system.

4 Experiments

Table 4.2: Query latency decomposed into local encoding, FAISS search, and external network time, averaged over 20 benchmark queries on an NVIDIA A100 GPU. Network time reflects internet latency to OpenAI servers and is not a property of the model.

Method	Encode (ms)	Search (ms)	Network (ms)	Total (ms)	Self-hostable
BM25	0.1	104.0	0.0	104.1	Yes
MiniLM	68.9	12.0	0.0	80.9	Yes
BGE	16.6	27.8	0.0	44.4	Yes
OpenAI	<i>server</i>	42.9	211.7	254.6	No

MiniLM encodes slowly despite being a small model. MiniLM’s 68.9 ms encoding time is slower than BGE’s 16.6 ms, despite having 15 times fewer parameters. This is because MiniLM encodes on CPU in these experiments, while BGE encodes on GPU. Hardware assignment dominates over model size for single-query encoding latency.

OpenAI’s latency is dominated by network time. Of OpenAI’s total 254.6 ms, only 42.9 ms is FAISS index search; comparable to BGE’s 27.8 ms search time. The remaining 211.7 ms is the round-trip time of the Hypertext Transfer Protocol (HTTP) request to OpenAI’s servers. This latency is not a property of the embedding model; it reflects internet connectivity between the university cluster and OpenAI’s infrastructure, and would vary across deployment environments. For a production system at Istari, API-based encoding would introduce an irreducible network dependency and per-query cost, whereas BGE could be self-hosted with predictable latency.

Summary. Table 4.3 summarises the key trade-offs across all four methods.

Table 4.3: Summary comparison of all baseline retrieval methods.

Method	NDCG@10	Prec@10	Recall@1k	Latency	Cost	GPU req.
BM25	0.166	0.156	0.682	104 ms	Free	No
MiniLM	0.192	0.186	0.653	81 ms	Free	No
BGE	0.227	0.216	0.796	44 ms	Free	Yes (enc.)
OpenAI	0.227	0.224	0.803	255 ms	\$0.02/M tok.	No

BGE emerges as the recommended baseline for subsequent experiments: it matches OpenAI’s retrieval quality, is the fastest method in practice, is free, and can be fully self-hosted without API dependency. The results motivate the next experimental phase, in which a pre-filtering stage and hybrid retrieval system are built on top of the BGE baseline to further improve precision at low cut-offs and overall system efficiency.

4.6 Evaluation Protocol Change for Follow-Up Experiments

All experiments described in the remainder of this chapter (Sections 4.7–4.16) use a **relaxed pseudo-relevance definition**: a company counts as relevant if it appears anywhere in production’s **top-1 000** results for a query, rather than the top-100 threshold used in Section 4.1.2 and Table 4.1. The relaxed threshold was adopted so that reranking, fusion, and oracle-ceiling experiments would have enough relevant companies per query to produce meaningful comparisons over large candidate pools. As a direct consequence, the absolute NDCG/Precision/Recall values reported from here on are on a different (generally much higher) numerical scale than Table 4.1, and **are not directly comparable to it**. All comparisons in the following sections should be read only relative to one another, within the same section.

Reading Precision and Recall together. Under this relaxed protocol, every one of the 101 queries has exactly 1 000 relevant companies. This has a direct consequence for how Precision@ k and Recall@ k relate to one another at any cut-off $k \leq 1\,000$: since both metrics are computed from the same count (how many of the top- k retrieved companies are relevant), just divided by a different denominator,

$$\text{Recall}@k = \text{Precision}@k \times \frac{k}{1\,000},$$

so the two metrics are exactly proportional to one another at every cut-off, and neither is redundant on its own: Precision@ k answers “of what the system returned, how much was correct”, while Recall@ k answers “of everything that is correct, how much did the system return”, and the two only coincide once $k = 1\,000$, where the denominators become equal. A practical consequence is that Recall@ k for small k can look misleadingly low in isolation. For example, a system that returns 10 companies of which 8 are genuinely relevant has an excellent Precision@10 of 0.80, but the corresponding Recall@10 is only $0.80 \times 10/1\,000 = 0.008$, not because the 10 companies returned are poor, but because 10 items can never cover more than 1% of a 1 000-company relevant set. For this reason, the tables in this chapter report Precision@ k at the shallow cut-offs ($k \in \{10, 50, 100, 300, 500\}$), where it is the more directly interpretable number, and report both Precision@1 000 and Recall@1 000 together, since $k = 1\,000$ is the one cut-off at which Recall stops being a rescaled copy of Precision and becomes a genuinely independent measure: total coverage of the full relevant set. Recall@1 000 is treated as the headline metric throughout this thesis for exactly this reason.

4.7 Extended Baseline Comparison: Additional Embedding Models

Five further dense embedding models were evaluated on the same 98 716-company corpus, all-fields text representation, and 101-query evaluation set described in Section 4.1.1:

4 Experiments

- **BGE-M3** (BAAI/bge-m3): a multi-lingual, multi-functionality embedding model, 1 024-dimensional embeddings.
- **GTE-large** (thenlper/gte-large): a general-purpose text embedding model, 1 024 dimensions.
- **E5-Mistral-7B-Instruct** (intfloat/e5-mistral-7b-instruct): an instruction-tuned 7-billion-parameter Mistral-based embedding model, 4 096 dimensions.
- **Linq-Embed-Mistral** (Linq-AI-Research/Linq-Embed-Mistral): a 7B-parameter Mistral-based embedding model fine-tuned for retrieval, 4 096 dimensions.
- **SFR-Embedding-Mistral** (Salesforce/SFR-Embedding-Mistral): Salesforce’s 7B-parameter Mistral-based retrieval embedding model, 4 096 dimensions.

Table 4.4: Extended baseline Recall@ k under the relaxed (top-1 000) pseudo-relevance protocol (Section 4.6). Bold indicates the best value in each column.

Method	Precision@ k				Recall@1000
	10	50	100	1000	
BGE-M3	0.927	0.916	0.905	0.704	0.704
GTE-large	0.946	0.936	0.925	0.768	0.768
E5-Mistral-7B	0.629	0.572	0.523	0.274	0.274
Linq-Mistral	0.934	0.921	0.915	0.749	0.749
SFR-Mistral	0.933	0.922	0.915	0.746	0.746

Findings. GTE-large is the strongest of the five additional models (Recall@1000 = 0.768, NDCG@10 = 0.946), ahead of Linq-Mistral (0.749 / 0.936), SFR-Mistral (0.746 / 0.938), and BGE-M3 (0.704 / 0.935). These four sit close together and clearly ahead of E5-Mistral-7B-Instruct, which substantially underperforms (Recall@1000 = 0.274, NDCG@10 = 0.631) despite sharing the same base architecture and parameter count as Linq-Mistral and SFR-Mistral. This indicates that fine-tuning data and strategy, not model size, are the dominant factor for this retrieval task. Notably, the two much smaller 1 024-dimensional models (BGE-M3, GTE-large) are competitive with or better than the 7B-parameter models, at a fraction of the encoding cost.

Encoding cost. The three 7B-parameter models required substantially more compute to encode the corpus: E5-Mistral took 2 011 seconds (33.5 min), Linq-Mistral 2 039 seconds (34.0 min), and SFR-Mistral 2 032 seconds (33.9 min), all on an A100 GPU, and all requiring checkpoint/resume logic to fit within the cluster’s 30-minute job wall-time limit. This is roughly five times the encoding time of the original 335M-parameter BGE-large baseline (7.0 minutes, Section 4.3.2), for retrieval quality that is not better and, in E5-Mistral’s case, substantially worse.

Incomplete and abandoned models. Two further large embedding models were attempted but did not produce final results. **BGE-Multilingual-Gemma2** (BAAI/bge-multilingual-gemma2, 9B parameters) encoding is still in progress at the time of writing: model loading alone takes 15–27 minutes on this cluster’s shared storage (highly variable due to filesystem I/O contention), leaving little of the 30-minute job window for actual encoding, so the job must be resubmitted repeatedly to make incremental progress via checkpointing. **NV-Embed-v2** (nvidia/NV-Embed-v2) was abandoned after a persistent `AttributeError` caused by an incompatibility between the model’s custom code and the installed `transformers` library version, which could not be resolved without a library downgrade that risked breaking the other baselines’ environment.

4.8 BGE Instruction-Prefix Correction

The original BGE-large baseline (Section 4.3.2) encoded queries without an instruction prefix. BGE-large’s model card specifies that, for retrieval tasks, each *query* should be prefixed with the instruction “*Represent this sentence for searching relevant passages:* ” before encoding; the passage (company) side requires no prefix, so the cached company embeddings could be reused without re-encoding the corpus. This fix was retested on both the all-fields and summary-only variants of the BGE baseline.

Table 4.5: Effect of the query instruction prefix on BGE-large, measured under the relaxed top-1 000 protocol (Section 4.6).

Variant	NDCG@10	Recall@1000	Δ NDCG@10	Δ Recall@1000
All-fields, no prefix	0.942	0.726	–	–
All-fields, with prefix	0.937	0.727	–0.005	+0.001
Summary-only, no prefix	0.937	0.700	–	–
Summary-only, with prefix	0.924	0.707	–0.014	+0.007

Finding. Contrary to expectation, applying the officially documented query prefix produced only a marginal and mixed effect on this dataset: Recall@1000 improved negligibly (+0.001 for all-fields, +0.007 for summary-only), while NDCG@10 slightly *decreased* in both variants. Both effects are small enough to plausibly be within query-level noise. This rules out the missing prefix as the explanation for a broader pattern seen throughout this chapter: under the relaxed protocol, BGE (0.726 all-fields, 0.700 to 0.707 summary-only) and the related BGE-M3 model (0.704, Section 4.7) both reach a lower Recall@1000 than the much smaller, general-purpose MiniLM model (0.741, Table 4.6), despite BGE’s larger parameter count and its retrieval-specific fine-tuning.

Why a specialised, larger model can still be a worse fit. A plausible explanation lies in what each model was optimised for during training, rather than in how it is queried at inference time. BGE-large and BGE-M3 are fine-tuned through hard negative mining

4 Experiments

(Section 4.3.2) on benchmark data dominated by short, single-topic passages retrieved for open-domain question answering, an objective that sharpens the model’s embedding space around the specific pattern of a question matched against a topically focused prose passage. MiniLM, by contrast, is trained as a general-purpose sentence-similarity model on a broad mixture of paraphrase and natural language inference data (Reimers and Gurevych 2019), an objective that rewards recognising that two pieces of text mean similar things across a wide range of domains and writing styles, rather than one narrow retrieval pattern. The company records retrieved in this thesis are not passages: each one concatenates heterogeneous, only loosely prose-like fields (organisation name, country, state, city, organisation type and size, industry code) with a short free-text summary. This structure is closer to the varied, mixed-format text that MiniLM’s training was designed to generalise across than to the passage-retrieval benchmarks that shaped BGE’s embedding space. Fine-tuning a model tightly around one task and one kind of text is well documented to improve performance on that specific benchmark while sometimes reducing robustness on text the model was not tuned for (Thakur et al. 2021), and the results in this chapter are consistent with that trade-off: BGE’s retrieval-specific specialisation does not transfer as cleanly to this heterogeneous company-record format as MiniLM’s broader, less specialised training does.

4.9 Query Expansion: HyDE Pipeline

HyDE (Hypothetical Document Embeddings) expands each query by asking a local LLM (llama3.1:8b via Ollama) to generate $N = 10$ stochastic (temperature = 0.8) hypothetical company descriptions that would ideally answer the query. The embeddings of these 10 descriptions are averaged into a single query vector, which is then searched against the MiniLM company index. A second stage reranks the top-50 HyDE results with a cross-encoder (BAAI/bge-reranker-v2-m3).

Table 4.6: HyDE pipeline results (relaxed top-1 000 protocol) compared against the plain MiniLM baseline re-evaluated under the same protocol.

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no expansion)	0.975	0.741	16.9
Stage 1: HyDE + MiniLM	0.829	0.596	9 246.9
Stage 2: HyDE + MiniLM + Reranker	0.954	0.596	9 488.3

Finding. Query expansion via HyDE *hurts* retrieval quality relative to directly encoding the original query: Stage 1 NDCG@10 drops from 0.975 to 0.829, and Recall@1000 drops from 0.741 to 0.596. Adding a cross-encoder reranker (Stage 2) recovers most of the NDCG@10 loss ($0.829 \rightarrow 0.954$) by re-ordering the top-50 candidates, but cannot recover the recall loss, since reranking only reorders an already -fixed candidate set. Latency is dominated overwhelmingly by the LLM call itself (9.2 seconds), three orders

of magnitude slower than directly encoding the query with MiniLM (16.9 ms). For this dataset, HyDE is not a worthwhile query-expansion strategy: it is far slower and retrieves a worse candidate set than simply embedding the query directly.

4.10 Hybrid Retrieval: Reciprocal Rank Fusion

As an alternative to query expansion, BM25 and MiniLM rankings are fused directly using Reciprocal Rank Fusion (RRF), $\text{RRF}(d) = \sum_{\text{method}} \frac{1}{k + \text{rank}(d)}$ with $k = 60$ (Cormack et al. 2009). A second stage reranks the top-50 fused results with the same cross-encoder used for HyDE.

Table 4.7: Hybrid RRF results (relaxed top-1 000 protocol).

Method	NDCG@10	Recall@1000	Latency (ms)
MiniLM (no fusion)	0.975	0.741	16.9
Stage 1: RRF (BM25 + MiniLM)	0.955	0.724	104.9
Stage 2: RRF + Reranker	0.975	0.724	352.0

Finding. Unlike HyDE, RRF fusion of BM25 and MiniLM adds negligible latency (the fusion step itself takes 2.2 ms) and, after reranking, matches the plain MiniLM baseline’s NDCG@10 (0.975) while Recall@1000 remains slightly below MiniLM alone (0.724 vs 0.741): BM25’s keyword matches do not add companies beyond what MiniLM already retrieves for this query set, so fusion mostly re-confirms rather than extends MiniLM’s candidate pool. RRF is a much cheaper and more effective way to incorporate a second retrieval signal than HyDE-style query expansion.

4.11 LLM-as-Judge Relevance Validation

Because the pseudo-relevance labels (Section 4.1.2) assume production’s own rankings are correct, an independent check was run using an LLM to directly judge relevance. For 20 held-out queries (from a pool of 101, split 81/20 train/held-out), the top-20 production results per query were judged individually on a 0/1/2 relevance scale by an LLM given the query and the company summary, yielding 400 graded judgements in total.

Finding. At $k = 10$, pseudo-relevance and LLM-judged NDCG agree closely (0.964 vs 0.963), supporting the use of production’s own top results as a reasonable relevance proxy at shallow depths. The two diverge sharply as k grows: LLM-judged NDCG stays essentially flat at 0.982 out to $k = 1000$, while pseudo-relevance NDCG falls to 0.771. This is consistent with the pseudo-relevance labels being a hard top-100 cutoff (Section 4.1.2): once retrieval looks past that cutoff, pseudo-relevance necessarily penalises

4 Experiments

Table 4.8: NDCG under pseudo-relevance labels vs. LLM-judged relevance labels, on the 20 held-out queries.

k	Pseudo-relevance NDCG	LLM-judged NDCG	Δ
10	0.964	0.963	-0.001
50	0.947	0.982	+0.035
100	0.939	0.982	+0.042
500	0.868	0.982	+0.113
1000	0.771	0.982	+0.210

the system even for companies an LLM judge considers genuinely relevant. This supports the interpretation, already flagged in Section 4.1.2, that pseudo-relevance NDCG at large k understates true system quality rather than reflecting a real quality gap.

4.12 Production API Evaluation and MiniLM Rerank

Two rounds of evaluation compared the local pipeline against Istari’s live production search API, using a separate ground truth (the production system’s own graded semantic-search results at a similarity threshold of 0.75) rather than the pseudo-relevance labels used elsewhere in this chapter.

v1: keyword API. The first round compared Istari’s v1 (BM25-based) production API against local MiniLM, both queried against the same 101 queries. Istari v1 BM25 NDCG@10 was 0.101 and Recall@1000 was 0.109 (network latency 2 828 ms/query), against local MiniLM’s NDCG@10 of 0.935 and Recall@1000 of 0.766 (local GPU latency 26.6 ms/query), confirming that the production keyword API is a much weaker retrieval signal than even the simplest local dense baseline.

v2: semantic API. A second round added Istari’s newer v2 semantic search API (live, paginated over the full 20-million-company GOI, quota-aware). Istari v2 NDCG@10 was 0.985 and Recall@1000 (at the achievable depth of 816 results) was 0.650, clearly ahead of local MiniLM (NDCG@10 0.935, Recall@816 0.709) at shallow cutoffs, though local MiniLM’s Recall catches up and slightly exceeds v2’s at $k = 816$. Network latency for v2 (1 966 ms/query for a full fetch) remains far higher than local MiniLM (26.4 ms/query).

Reranking v2’s own candidates with MiniLM. A follow-up experiment took v2’s own retrieved candidate pool and re-ordered it locally using MiniLM, to test whether local reranking could improve on v2’s native ranking without changing which companies are retrieved. This made results *worse*: NDCG@10 fell from 0.985 (v2 native ranking) to 0.855 (MiniLM-reranked). Istari’s v2 semantic ranking is not simply a MiniLM-style dense embedding search under the hood: it outperforms MiniLM’s own ranking of the same candidates.

4.13 ANN Index Scaling to 2 Million Vectors

To test whether the exact search used throughout this chapter (`IndexFlatL2/IndexFlatIP`) remains practical at production scale, the 98 716 real MiniLM company embeddings were padded with 1 901 284 synthetic filler vectors (real embeddings perturbed with Gaussian noise at 15% of the average vector norm), giving a 2-million-vector corpus. Exact search (`FlatL2`) was compared against approximate indexes: IVF (`nlist=4096`, with `nprobe` $\in \{1, 8, 32, 128, 4096\}$) and HNSW ($M = 32$, with `efSearch` $\in \{16, 64, 256\}$).

Table 4.9: Exact vs. approximate search at 2M-vector scale (relaxed top-1 000 protocol). Only representative configurations shown.

Index	NDCG@10	Recall@1000	ms/query	Speedup	Index size
Exact (FlatL2)	0.975	0.741	175.5	1.0×	3 072 MB
IVF (<code>nprobe=8</code>)	0.974	0.743	26.4	6.6×	3 094 MB
HNSW (<code>efSearch=256</code>)	0.975	0.727	26.9	6.5×	3 616 MB

Finding. Both IVF (`nprobe=8`) and HNSW (`efSearch=256`) recover essentially all of exact search’s NDCG@10, and IVF slightly exceeds exact search’s Recall@1000 (0.743 vs 0.741, within noise), while running 6.5–6.6× faster at 2-million-vector scale, at the cost of modest additional index memory (0.7% and 17.7% larger, respectively). This confirms that approximate search is a safe and substantial efficiency gain for scaling the retrieval system toward the full 20-million-company GOI, without a meaningful quality trade-off at this scale.

4.14 Recall@1000 Error Analysis

To understand why exact MiniLM search caps at Recall@1000 = 0.741 (Section 4.13), every production-relevant company that MiniLM failed to retrieve within the top-1 000 was extracted and searched for at much greater depth ($k = 10\,000$), yielding 26 185 missed (query, company) pairs across all 101 queries.

Table 4.10: Classification of the 26 185 Recall@1000 misses by how far beyond rank 1 000 the true company was actually ranked.

Bucket	Count	Share (%)
Near miss (rank 1 000–2 000)	16 157	61.7
Mid miss (rank 2 000–5 000)	8 061	30.8
Far miss (rank 5 000–10 000)	1 375	5.3
Not found (beyond rank 10 000)	592	2.3

Finding. The large majority of misses (61.7%) are near misses, just outside the top-1 000 cutoff, suggesting most of the recall gap is a matter of ranking granularity rather than a hard semantic failure. Only 2.3% of misses are not found at all within $k = 10\,000$, indicating genuine embedding failures are rare. Separately, 3.8% of missed companies (996 of 26 185) were flagged as boilerplate content, and one extreme case of description duplication was found: the placeholder text “Bluehost.com” appears as the company description for 709 distinct domains, meaning these domains are indistinguishable to any text-embedding model and can never be individually retrieved correctly. This is a concrete data-quality artifact rather than a retrieval-model limitation, and sets a hard ceiling on achievable recall for any embedding-based method operating on the `summary` field alone.

4.15 Corpus Quality: Platform Boilerplate in the AI-Generated Summary

The duplicated “Bluehost.com” description found in Section 4.14 is one instance of a broader, previously unquantified data-quality issue affecting the `summary` field that every retrieval method in this thesis depends on. For a meaningful share of companies in the corpus, the AI-generated summary describes the website hosting provider or website-building platform that the company’s domain happens to run on, rather than the company’s own business. This happens because many small businesses have a thin, template-driven web presence, so the process that generates each company’s summary from its web content sometimes picks up generic platform language (for example, marketing copy for a web hosting provider or an e-commerce storefront builder) instead of any information specific to the company itself. A company using such a platform is then, in effect, indistinguishable in its summary text from every other company using the same platform, which is precisely the mechanism behind the large-scale description duplication already observed in Section 4.14.

How much of the corpus is affected. This contamination is not spread evenly across query topics. An audit of the pseudo-relevance labels for all 101 queries found that, for the query “*web hosting companies*”, 85.4% of the 1 000 ground-truth relevant companies have a boilerplate rather than a company-specific summary; for “*cloud computing providers*” the figure is 73.1%, and for “*e-commerce platforms*” it is 53.1%. By contrast, most other query topics in the evaluation set (among them “*software companies*”, “*healthcare providers*”, and “*construction companies*”) show no measurable contamination at all. This pattern is intuitive once the underlying mechanism is understood: boilerplate summaries describe hosting, domain, and e-commerce infrastructure, so they are only likely to be retrieved, and only likely to be judged relevant by the production system’s own ranking, for queries whose intent is itself adjacent to that infrastructure. A query about software companies is unlikely to surface a domain registrar’s boilerplate text, but a query about web hosting companies is exactly the case in which that boilerplate text looks, on the surface, like a plausible match.

An attempt to detect it automatically. A classifier was built to flag boilerplate summaries automatically, using embedding similarity between each company’s summary and a small, fixed reference set of around 300 known boilerplate templates, an approach chosen so that classifying any single company costs one embedding comparison against a fixed-size reference set, regardless of whether the underlying corpus is the 98 716-company evaluation set used in this thesis or the full multi-million-company GOI. This classifier was validated against 300 companies independently judged by an LLM as having a trustworthy or untrustworthy summary. At its best-performing similarity threshold, the classifier reaches an F1 score of 0.59, with a precision of 0.52 and a recall of 0.69 for detecting untrustworthy summaries.

Why this limitation was not corrected in the reported results. A precision of 0.52 means that of every two summaries the classifier flags as boilerplate, roughly one is, in fact, a genuine, company-specific summary. Removing or down-weighting every flagged company from the retrieval pipeline would therefore discard a large number of companies that are not actually contaminated, trading one source of error for another rather than removing it. For this reason, no correction for boilerplate contamination was applied to any of the results reported in this thesis, and all recall and precision figures should be read with this limitation in mind, particularly for the small set of query topics identified above as heavily affected. This is reported here as an explicitly known and quantified, but unaddressed, limitation of the underlying corpus rather than as a solved problem: a natural direction for future work is to use the boilerplate-similarity score not as a hard filter but as an additional input feature to a trained model, such as the learned fusion ranker introduced in Section 4.21, which is able to learn how much to trust an imperfect signal rather than requiring it to be reliable enough for a fixed decision rule.

4.16 Oracle Union Recall Ceiling

Before investing in a trained reranker or a fine-tuning pipeline, it is useful to know the best-case recall achievable by simply combining the candidate pools already produced by every method run so far — at zero additional encoding cost, since every channel’s top-1 000 results per query are already cached on disk. This experiment computes, for every combination of the six available channels (MiniLM, BM25, BGE-all-fields, BGE-summary, OpenAI-large, Nomic), the recall of the *union* of their top-1 000 candidate sets against the top-1 000 ground truth.

Findings. The best single channel (MiniLM alone) reaches $\text{Recall@1000} = 0.741$. The full union of all six channels reaches 0.936 (average combined pool size 1 949 candidates/query), a gain of +0.196 over MiniLM alone. The best-performing *pair* is MiniLM + OpenAI-large, at 0.856 (average pool size 1 300). A pool-size sweep on this best pair shows the ceiling is highly sensitive to how many candidates are taken per channel: at 100 candidates/channel the union only reaches 0.150 recall, rising to 0.581 at 500, and 0.856 at the full 1 000.

Interpretation. Because the six-channel union clears 0.936, well above any individual channel, the different retrieval methods are largely making *complementary*, not correlated, errors: a reranker operating over a modestly sized union pool (rather than a single channel) is likely sufficient to approach this ceiling, and a full fine-tuning pipeline is not obviously necessary to close the remaining gap. The residual 0.064 recall lost even under the full six-channel union is consistent with the hard data-quality ceiling identified in Section 4.14 (duplicated/boilerplate company descriptions that no embedding-based channel can disambiguate).

4.17 SimLM: A Purpose-Built Retrieval Pre-training Baseline

All dense baselines evaluated so far are general-purpose sentence-embedding models. **SimLM** (Wang et al. 2022) takes a different approach: it is a BERT-base (110 M parameter) encoder pre-trained specifically for passage retrieval using a representation-bottleneck objective, before any task-specific fine-tuning. It is included to test whether a model purpose-built for retrieval, but far smaller than the 7B-parameter models in Section 4.7, can match or exceed them.

Implementation. The public checkpoint `intfloat/simlm-base-msmarco-finetuned` is used, following its documented convention rather than the all-fields concatenation used elsewhere in this thesis: the company name is encoded as a *title*, and the remaining fields (country, state, city, type, size, industry, summary, keywords) as the paired *body* text, using CLS-token pooling (not mean pooling) with L2 normalisation. Queries are encoded without any prefix, following the same model card.

Table 4.11: SimLM Recall@1000 (relaxed top-1 000 protocol) alongside the strongest baselines evaluated so far, for reference.

Method	Precision@ <i>k</i>				Recall@1000
	10	50	100	1000	
BM25 (Section 4.2)	0.838	0.748	0.723	0.635	0.635
SimLM	0.920	0.881	0.861	0.609	0.609
MiniLM (Section 4.3.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.7)	0.946	0.936	0.925	0.768	0.768

Finding. SimLM underperforms every dense baseline evaluated in this thesis, and its Recall@1000 (0.609) falls even below the lexical BM25 baseline (0.635), despite SimLM being purpose-pretrained for passage retrieval. (NDCG@10 tells a less clear-cut story here: SimLM’s 0.923 sits above BM25’s 0.854, since NDCG rewards ranking quality among whichever companies are retrieved, not coverage of the full relevant set.) A likely explanation is a domain mismatch between SimLM’s pre-training convention and

this task: SimLM was pre-trained and fine-tuned on MS MARCO, a corpus of short, single-topic passages, using a title/body sentence-pair format that assumes the title is a concise topical label. Company names are not topical labels in the same sense: “Sweet Software” or “Springcoast Capital Partners” convey little about what the paired body text describes, so the sentence-pair convention that helps SimLM on MS MARCO-style data may provide a weaker signal here than simply concatenating all fields into one string, as the other baselines do. This result is a useful negative finding: retrieval-specific pre-training does not automatically transfer across domains, and general-purpose sentence embeddings (MiniLM, GTE-large) remain the stronger choice for this task.

4.18 ColBERT: Late-Interaction Retrieval from Scratch

Every method evaluated so far represents each company and each query as a single dense vector. **ColBERT** (Khattab and Zaharia 2020; Santhanam et al. 2022) instead represents both as *sets of per-token* embeddings and scores a (query, company) pair using *MaxSim*: for every query token, the maximum similarity to any company token is taken, and these per-token maxima are summed. This is architecturally richer than single-vector dense retrieval, since no single fixed-size vector has to compress the entire meaning of a company description, and it is included to test whether that added expressiveness translates into higher recall on this task.

Implementation. The public checkpoint `colbert-ir/colbertv2.0` is used, encoding each company summary into a sequence of 128-dimensional per-token embeddings via the model’s linear projection layer. Due to indexing and library constraints on the cluster used for this thesis, the official PLAID-indexed ColBERT pipeline was not used; instead, MaxSim scoring is implemented from scratch as a brute-force computation over all 98 716 companies for every query. This is computationally more expensive than an indexed implementation would be, but is exact rather than approximate, and is tractable at this corpus size.

Table 4.12: ColBERT (brute-force MaxSim) Recall@1000 (relaxed top-1 000 protocol) alongside the strongest single-vector baselines.

Method	Precision@ <i>k</i>				Recall@1000
	10	50	100	1000	
MiniLM (Section 4.3.1)	0.972	0.950	0.938	0.741	0.741
GTE-large (Section 4.7)	0.946	0.936	0.925	0.768	0.768
ColBERT (MaxSim)	0.936	0.931	0.925	0.773	0.773

Finding. ColBERT’s Recall@1000 (0.773) is the highest of any single retrieval channel evaluated in this thesis, narrowly ahead of GTE-large (0.768) and clearly ahead of

MiniLM (0.741). Its NDCG@10 (0.937), however, is lower than both MiniLM (0.975) and GTE-large (0.946), meaning ColBERT retrieves a broader set of relevant companies overall but ranks the very top results somewhat less precisely. Token-level interaction therefore does deliver a genuine, if modest, recall improvement over the best single-vector models, though not one large enough on its own to close the gap to the hybrid rerank pipelines described in Section 4.19.

4.19 Hybrid Full-Pool Reranking

Section 4.10 showed that fusing BM25 and MiniLM with Reciprocal Rank Fusion, followed by reranking only the top-50 fused candidates, essentially reproduces MiniLM’s own quality (NDCG@10 = 0.975) without improving Recall@1000. Section 4.16 then showed that the *union* of several channels’ top-1 000 candidate pools reaches a much higher oracle recall ceiling than any single channel. Together, these two findings motivate a different hybrid design: rerank the *entire* union pool of several dense channels with a cross-encoder, rather than fusing rankings and reranking only a shallow, fixed-size slice of the result.

Implementation. For each query, the top-1 000 results from a set of dense channels are combined by simple set union (no rank fusion, no truncation), giving an unranked candidate pool that is typically 1 300–1 800 companies. A cross-encoder reranker, `BAAI/bge-reranker-v2-m3` (BAAI 2024), then scores every (query, candidate) pair in the full pool directly, and the top-1 000 by reranker score are kept as the final result. Four variants are compared, adding one channel at a time:

- **2-way:** MiniLM + Linq-Embed-Mistral.
- **3-way:** 2-way + GTE-large.
- **4-way:** 3-way + OpenAI-large.

Table 4.13: Full-pool reranking (relaxed top-1 000 protocol), compared against the shallow RRF + top-50-rerank baseline from Section 4.10. Bold indicates the best value.

Method	P@10	P@100	P@1000	R@1000	Pool
RRF + top-50 (§4.10)	0.980	0.940	0.782	0.782	50
2-way (MiniLM+Linq)	0.955	0.950	0.802	0.802	1 300
3-way (+GTE-large)	0.955	0.950	0.804	0.804	1 424
4-way (+OpenAI-large)	0.955	0.950	0.802	0.802	1 600

Finding. Reranking the full union pool instead of a fixed top-50 slice raises Recall@1000 substantially ($0.782 \rightarrow 0.802$, 2-way), at the cost of a small drop in NDCG@10 ($0.979 \rightarrow 0.955$): the reranker now has many more genuinely difficult candidates to rank, so its ordering of the very top results is slightly less precise, even though it recovers more of the true relevant set overall. Adding a third channel (GTE-large) improves Recall@1000 further, to 0.804, the best result obtained by any hybrid pipeline built from off-the-shelf pretrained components in this thesis. Adding a fourth channel (OpenAI-large) does *not* improve on this – Recall@1000 is unchanged at 0.802, within noise of the 3-way result.

Interpretation: channel saturation. This flat 3-way $\rightarrow$ 4-way result is an important turning point in this thesis’s experimental narrative. Section 4.16 showed that combining *more* candidate-generating channels reliably raises the oracle recall ceiling; here, adding a fourth channel to an already capable 3-way pool does not translate into a better final result. This suggests that candidate coverage is no longer the limiting factor for this pipeline: the union pool already contains the relevant companies needed to reach a substantially higher recall than 0.804 (recall Section 4.16’s six-channel oracle ceiling of 0.936) – and the reranker itself is now the bottleneck preventing the pipeline from closing that gap. This motivates two follow-up questions, addressed in the next two sections: is the specific reranker *model* the limiting factor (Section 4.20), or is a fundamentally different fusion approach needed (Section 4.21)?

4.20 Reranker Ablation: Isolating the Bottleneck

Given the channel-saturation finding in Section 4.19, the natural next question is whether the specific reranker model used throughout this chapter, BAAI/bge-reranker-v2-m3, is itself limiting recall, or whether reranker choice matters little once the candidate pool is fixed.

Implementation. BAAI/bge-reranker-v2-m3 (built on an XLM-RoBERTa backbone) is replaced with mixedbread-ai/mxbai-rerank-large-v1 (Mixedbread AI 2024) (built on a DeBERTa-v2 backbone, an architecturally distinct, comparably-sized cross-encoder), applied to the *exact same* 3-way union pool from Section 4.19. Only the reranker model changes; the candidate pool, corpus, and queries are identical.

Table 4.14: Reranker ablation Recall@1000: identical 3-way candidate pool, two different reranker models (relaxed top-1 000 protocol).

Reranker	P@10	P@100	P@1000	R@1000
BAAI/bge-reranker-v2-m3 (§4.19)	0.955	0.950	0.8042	0.8042
mxbai-rerank-large-v1	0.968	0.940	0.8035	0.8035

Finding. Recall@1000 is essentially unchanged between the two rerankers (0.8042 vs. 0.8035, a difference of 0.0007, well within query-level noise). NDCG@10 does improve modestly with the alternative reranker (0.955 $\rightarrow$ 0.971), indicating that `mxbai-rerank-large-v1` orders the very top results somewhat better, without changing which companies land in the top-1000 overall. The conclusion is that reranker *model choice* is not the bottleneck identified in Section 4.19: the remaining gap to the oracle ceiling is not a matter of picking a better off-the-shelf cross-encoder. This points towards a more fundamental limitation: either genuinely hard, vocabulary-mismatched cases that no cross-encoder scoring the `summary` field alone can resolve, or a fusion mechanism (simple set union plus independent reranking) that is itself insufficiently expressive, motivating the learned approach in Section 4.21.

4.21 Learned Fusion Ranker

Every hybrid pipeline so far combines *frozen*, independently pretrained components: candidate lists are pooled by simple union, and a pretrained cross-encoder, never fit to this task’s own queries or labels, reranks the result. This section instead treats fusion as a supervised learning-to-rank problem (Liu 2009): a model is trained directly on this thesis’s own 101 queries and their pseudo-relevance labels to learn how to combine multiple retrieval signals, rather than relying on a fixed, hand-designed combination rule.

Implementation. For every (query, candidate) pair in the same 3-way union pool used in Section 4.19 (with BM25 added as a fourth, lexical, signal), ten features are computed: the raw similarity score and inverse rank (1/rank) from each of MiniLM, Linq-Embed-Mistral, GTE-large, and BM25; the `BAAI/bge-reranker-v2-m3` score from Section 4.19 (included as one input feature among ten, not as the final decision rule); and a count of how many channels retrieved the candidate at all. Two models are trained on these features using `scikit-learn` (Pedregosa et al. 2011): a logistic regression, and a histogram-based gradient-boosted tree ensemble. Crucially, both are trained and evaluated using 5-fold cross-validation grouped by `query_id`: for each fold, the model is fit on candidates from 80% of queries and scores held-out candidates from the remaining 20%, so that no query is ever scored by a model that has seen its own label during training. Final metrics are computed by concatenating the held-out predictions from all five folds, giving a complete, leakage-free evaluation over all 101 queries.

Finding. The gradient-boosted learned fusion ranker reaches Recall@1000 = 0.823, the best result obtained anywhere in this thesis, improving on the best pretrained-component pipeline (0.804) by +0.019. For scale, this gain is roughly 27 times larger than the 0.0007 movement observed when merely swapping reranker models in Section 4.20; this is a substantive improvement attributable to training on the task’s own data, not noise. NDCG@10 also improves (0.955 $\rightarrow$ 0.979).

Table 4.15: Learned fusion ranker Recall@1000 (held-out, cross-validated predictions; relaxed top-1 000 protocol), compared against the best pretrained-component pipeline from Section 4.19. Bold indicates the best value.

Method	P@10	P@100	P@1000	R@1000
3-way + BGE reranker (§4.19)	0.955	0.950	0.804	0.804
Learned fusion, logistic regression	0.968	0.953	0.821	0.821
Learned fusion, gradient-boosted trees	0.977	0.955	0.823	0.823

Interpretation: what the model learned. The trained logistic regression’s coefficients offer a direct, interpretable view into which signals matter most for this task: MiniLM’s *inverse rank* has by far the largest coefficient magnitude (+5.01), exceeding every raw similarity score *and* the pretrained reranker’s own score (+0.16). In other words, where a candidate is ranked by MiniLM is a more informative signal for final relevance than the pretrained cross-encoder’s score: a concrete, non-obvious finding that would not be visible from simply combining components by hand. This supports the conclusion drawn in Section 4.20: the pretrained reranker was not extracting the maximum available signal from the candidate pool, and a lightweight model trained on this task’s own data closes a meaningful part of the remaining gap to the oracle ceiling from Section 4.16.

4.22 Validating the Learned Fusion Ranker Against a Non-Circular Gold Standard

The pseudo-relevance protocol used throughout this thesis (Section 4.1.2) treats a company as relevant if and only if it appears in production’s own results for a query. This is a reasonable practice when no human-annotated relevance judgements exist (Section 3.5), but it carries a specific risk for the learned fusion ranker (Section 4.21): a model trained directly on labels derived from production’s own output may partly be learning to reproduce whatever idiosyncrasies that particular system has, rather than genuine relevance. This section reports a small-scale attempt to check whether the learned fusion ranker’s key interpretive finding, that MiniLM’s *inverse rank* is the most informative feature, survives replacing that circular ground truth with an independently constructed one.

4.22.1 Building a Pooled, Independently Judged Gold Standard

Pooling. Following the pooling methodology established for the Text REtrieval Conference (Voorhees 1999; Zobel 1998), the top-20 candidates per query were taken from six sources: the frozen production ground truth, a freshly fetched sample from Istari’s live v2 search API, BM25, GTE-large, ColBERT, and the learned fusion ranker’s own output, then unioned and deduplicated by domain. Applied across all 101 queries, this yields 8 309 pooled candidates, an average of 82.3 unique candidates per query, a substantially smaller and more tractable set than the original 1 000-per-query pool.

4 Experiments

Trust filtering. Before any relevance judging, every pooled candidate’s summary was checked against the boilerplate detector from Section 4.15, reusing its already-cached reference templates. This flagged 487 of 8 309 candidates (5.9%) as untrustworthy, which were excluded from judging entirely, so that a summary describing the wrong entity could not poison a relevance label built on top of it.

Multi-judge relevance labelling. Each remaining, trustworthy candidate was independently judged by two large language models (OpenAI’s `gpt-4o-mini` and Anthropic’s Claude Haiku), on a three-level scale (0 = not relevant, 1 = partially relevant, 2 = highly relevant), following the LLM-as-judge approach validated at scale by Thomas et al. (2024). Where the two judges agreed, that label was accepted directly; where they disagreed, the case was resolved by manual review.

Blind calibration of the judges. Resolving disagreements by reading both judges’ stated reasoning risks anchoring the human reviewer toward whichever judge’s framing is read first. To check this, a separate blind sample of 20 candidates that neither judge had yet seen was labelled by the author with no access to either judge’s output, then scored independently by both judges. OpenAI matched the blind label 75% of the time and Claude 80%, close enough that neither judge should be treated as categorically more reliable than the other on this task.

Scope of this pilot. Labelling all 101 queries with two paid LLM judges was not pursued in the time available; instead, a 5-query pilot was completed first. Of 430 candidates judged, 360 (83.7%) received unanimous agreement between the two judges. Of the remaining 70 disagreements, 59 were resolved by manual review and 11 were excluded outright for insufficient evidence (for example, unreachable domains or placeholder websites with no verifiable content), rather than being forced into a label the available information could not support. This yields **419 gold-labelled candidates** across 5 queries, distributed as 8 not relevant, 119 partially relevant, and 292 highly relevant.

4.22.2 Retraining Against the New Gold Standard

The exact architecture from Section 4.21 (the same ten features, the same 5-fold cross-validation grouped by `query_id`, here equivalent to leave-one-query-out since exactly five queries are covered) was retrained using the 417 of 419 gold-labelled candidates that matched an existing feature row from the original candidate pool. The binary classification target used the strict threshold `gold_label = 2` (highly relevant only, 291 positive vs. 126 negative examples); the more permissive threshold `gold_label ≥ 1` leaves only 8 negative examples across the whole pilot, too few for cross-validated coefficients to be estimated reliably.

Finding. Retraining on the pooled, independently judged gold standard modestly improves ranking quality over simply reusing the original model’s scores (NDCG@5: 0.915 → 0.934;

4 Experiments

Table 4.16: NDCG@ k against the new gold standard: the retrained model compared against the original learned fusion ranker’s held-out scores, both evaluated on the same 417 candidates.

Model	NDCG@5	NDCG@10	NDCG@20
Original model’s scores (§4.21)	0.915	0.930	0.940
Retrained on new gold standard	0.934	0.952	0.947

Table 4.17: Logistic regression feature weights across three training sets: the original (circular) ground truth, the 5-query pilot pooled gold standard, and the expanded gold standard covering all 101 queries (Section 4.23.1).

Feature	Original	Pilot (5 queries)	Expanded (101 queries)
invrank_minilm	+5.014	−0.909	−1.095
score_linq	+1.097	+1.965	+1.381
invrank_gte	+0.958	+0.323	+0.072
score_gte	+0.921	+0.791	−0.701
invrank_linq	+0.867	+0.416	+0.172
score_minilm	+0.749	−0.146	−0.703
n_channels	+0.459	−0.289	+0.394
invrank_bm25	−0.258	−1.252	−2.246
reranker_score	+0.164	+0.319	+0.146
score_bm25	+0.054	−0.134	−0.069

NDCG@10: 0.930 $\rightarrow$ 0.952), confirming that the new labels carry a genuine, learnable signal rather than just noise. More importantly, the original interpretive finding from Section 4.21 does not replicate: `invrank_minilm`, previously the dominant feature by a wide margin (+5.014, more than double the next-largest coefficient), collapses not merely toward zero but to a small *negative* coefficient (−0.909). The most plausible explanation is that MiniLM’s ranking correlated strongly with production’s own circular output specifically, rather than with relevance itself, a correlation that a model trained and evaluated against that same circular output could not distinguish from a genuine relevance signal. `score_linq` (Linq-Embed-Mistral’s raw similarity score) is the new dominant feature (+1.965), and `invrank_bm25` is now strongly negative (−1.252): being ranked highly by BM25 is associated with *lower* true relevance in this sample. This corroborates, independently and statistically, the qualitative BM25 failure mode already documented in Section 4.2.2, where investment firms and venture capital companies are ranked highly by BM25 for the query “software companies” because they mention the word in passing while describing their investment portfolio, without themselves being software companies.

Confirmation at full scale. The Limitations paragraph below originally flagged this as a 5-query pilot whose specific coefficients should be read cautiously. Section 4.23.1’s

4 Experiments

expanded gold standard (1,429 candidates across all 101 queries) makes it possible to check this directly: retraining the same architecture on the full 101-query gold standard (rightmost column of Table 4.17) does not weaken the finding, if anything it sharpens it. `invrank_minilm` remains negative (-1.095 , was -0.909 at 5 queries), and `invrank_bm25` becomes even more strongly negative (-2.246 , was -1.252), now the single largest-magnitude coefficient in the model. `score_linq` remains the dominant positive feature throughout all three columns. The BM25 vocabulary-mismatch failure mode is therefore not an artefact of the “software companies” pilot query specifically, it holds consistently once every one of the 101 query topics contributes labelled examples.

Limitations. This result covers 5 of 101 queries and 417 candidates, a genuine pilot rather than a full replication, and the specific numerical coefficients should be read with that scale in mind. However, the magnitude of the reversal on what was previously the single dominant feature is unlikely to be an artefact of sample size alone, and is treated here as a preliminary but suggestive finding: the interpretive claims drawn from a model trained on circular pseudo-relevance labels should not be taken as descriptions of genuine relevance without independent validation. Scaling this pooled, multi-judge gold standard to the full 101-query set is a direction for future work.

4.23 Scaling Beyond the Gold Standard: Silver Labels via Model Distillation

Extending independent LLM judging to every candidate the retrieval systems in this thesis actually produce is not affordable. The original fusion ranker’s own candidate pool (Section 4.21) covers 173 262 (query, candidate) pairs across all 101 queries; judging every one of those with two LLM judges, at the per-candidate cost observed in Section 4.22.1, would cost approximately \$22 at the current pooling depth, or as much as \$28 000 if every one of the 98 716 corpus companies were judged against every one of the 101 queries. The latter figure is not just expensive but methodologically wasteful: the overwhelming majority of those pairs are companies that no retrieval channel ever considered plausible for that query, judging them would mean paying to confirm the obvious.

Approach. Instead, the 419 gold-labelled candidates from Section 4.22.1 are used to train a final classifier (the same architecture and feature set as Section 4.21, fit on all 419 labels rather than held out via cross-validation, since this model is intended for deployment rather than evaluation). This classifier is then applied to all 173 262 existing candidate rows, which already have the required ten features computed, at no additional retrieval or labelling cost. This produces two explicitly distinct label tiers, following standard practice in weak and distant supervision: a small **gold** tier (419 candidates, LLM-judged and human-adjudicated) and a much larger **silver** tier (the remaining candidates, model-predicted rather than independently judged). The two tiers are not merged or treated as equally reliable anywhere in this pipeline.

Validating the silver tier. To estimate how trustworthy the silver labels actually are, a random sample of 150 silver-labelled candidates is independently judged by two more capable models, `claude-sonnet-5` and `gpt-5.4` (an upgrade from the `claude-haiku-4-5/gpt-4o-mini` pair used for the original gold standard, chosen after confirming via the blind calibration check in Section 4.22.1 that the cheaper pair was not itself the limiting factor). Only candidates where both stronger judges unanimously agree are counted, disagreements are excluded from the spot-check rather than guessed at, the same principle used throughout Section 4.22.1. This validation costs under \$1, compared to the \$22 000+ a full re-judging pass would require.

Status. The silver-labelling classifier has been trained and applied to all 173 262 candidates. Of the 150 spot-checked silver candidates, 132 (88%) received unanimous agreement between `gpt-5.4` and `claude-sonnet-5`; against that unanimous subset, the silver labels achieved 65.9% accuracy (precision on the relevant class 65.3%, recall 85.3%). A naive baseline that always predicts the majority class on this same 132-candidate sample would score 56.8%, so the silver labels carry a real, learnable signal, but a modest one: the classifier over-predicts relevance (34 false positives against 11 false negatives out of 132), consistent with the 75.4% overall relevant-rate it assigns across the full silver tier.

The most likely explanation for the gap between this figure and the substantially higher NDCG improvement reported in Section 4.22.2 is a training-distribution mismatch rather than a flaw in the distillation approach itself: only 8 of the 132 unanimously-judged spot-check candidates (6%) fall under one of the 5 queries the underlying gold standard was built from, the remaining 124 (94%) come from query topics the classifier never saw a single labelled example of during training. Generalising from 5 narrow query topics to the other 96 is a substantially harder test than the leave-one-query-out cross-validation in Section 4.22.2, which only ever held out one of those same 5 topics at a time. This points toward a concrete, affordable fix rather than abandoning the approach: expanding the gold standard to a larger, more topically diverse subset of the 101 queries (at the roughly \$22 cost estimated above for full-depth pooling) before the resulting silver labels are relied on broadly, rather than distilling from 5 queries alone.

4.23.1 Expanding the Gold Standard via Active Learning

Rather than judging a random or exhaustive sample of the remaining 96 uncovered queries, the gold standard is expanded using *uncertainty sampling*: the distillation classifier’s own predicted probability is used to find, for every one of the 101 queries, the ten candidates it is least confident about (predicted probability closest to 0.5), and only those are sent for judging. This targets exactly the candidates most likely to teach the classifier something it does not already know, rather than spending judging budget on cases it already calls correctly. The judging prompt is enriched at the same time with company metadata that was previously collected but never shown to the judges: `organization_type`, `organization_size`, `state`, `district`, `municipality`, and `nace_code`, alongside the original name, country, and summary fields, giving the LLM judges more grounding for cases a summary alone leaves ambiguous. This active-learning approach is well estab-

4 Experiments

lished in the machine learning literature (Settles 2009), and combining LLM-based and human-provided relevance judgments to build a test collection under a limited labelling budget mirrors the approach of Takehi et al. (2024), who propose a similar strategy for balancing manual and LLM annotation effort in IR test collection construction.

Figure 4.1 summarises the full pipeline described across this section: pooling, multi-judge labelling, human tie-breaking, distillation into silver labels, the spot-check that revealed the out-of-distribution problem, and the active-learning step that expands the gold standard in response.

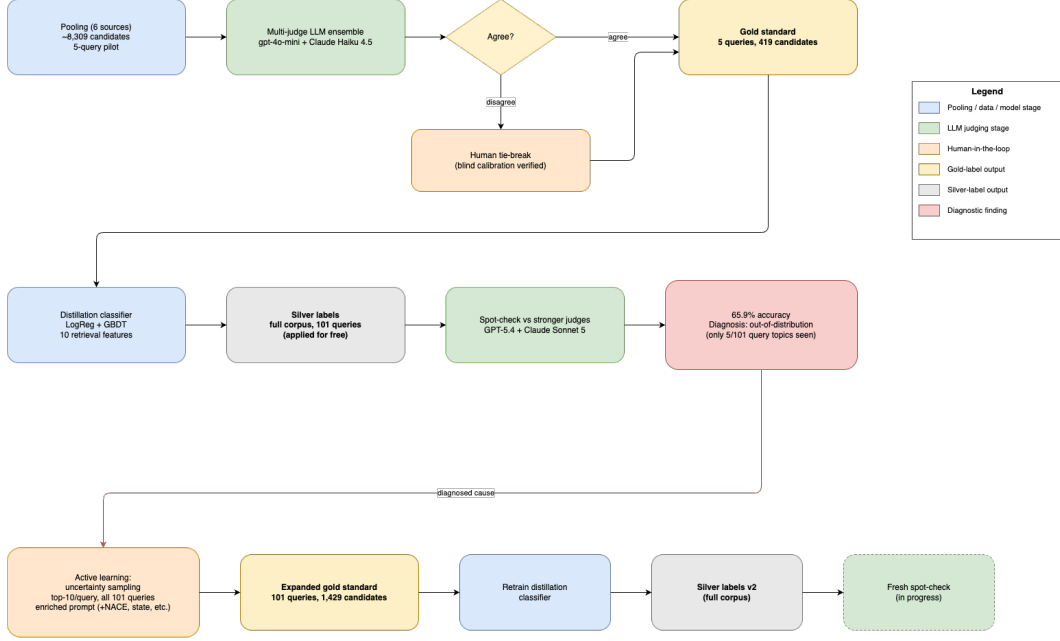


Figure 4.1: The gold and silver standard construction pipeline: pooling from six sources, multi-judge LLM labelling with human tie-breaking, distillation into silver labels at full scale, the spot-check that revealed poor out-of-distribution generalisation, and the active-learning step that expands gold-standard query coverage in response.

Result. This active-learning pass covers all 101 queries (versus the original 5) and adds 1,010 newly judged candidates, 852 (84.4%) agreed unanimously between the two judges, 158 needed human tie-breaking, a disagreement rate close to the original pilot’s 16% once an initial data-merging bug that had corrupted roughly 40% of the first labelling pass with a missing query field was found and fixed. Combined with the original 419, the expanded gold standard now contains **1,429 candidates spanning all 101 queries** (label distribution: 63 not relevant, 381 partially relevant, 985 highly relevant), a nearly $3.5\times$ increase in size and full query coverage instead of a 5% sample of it.

4 Experiments

Result. The distillation classifier was retrained on the full 1,427 gold-labelled candidates matched to features (spanning all 101 queries) and re-applied to all 173 262 candidates. A fresh 150-candidate spot-check was drawn (a different random sample than the first, so as not to reuse previously-checked candidates) and judged with the same stronger pair, `gpt-5.4` and `claude-sonnet-5`. Of these, 140 (93.3%) received unanimous agreement, slightly higher than the first spot-check’s 88%. Against that unanimous subset, silver-label accuracy rose from **65.9% to 80.7%**. The naive majority-class baseline also shifted with the new sample (56.8% $\rightarrow$ 64.3%), so the more informative comparison is the *margin* over that baseline: it nearly doubled, from 9.1 percentage points to 16.4. The earlier bias toward over-predicting relevance is substantially reduced: false positives fell from 34/132 (25.8%) to 11/140 (7.9%), precision on the relevant class rose from 65.3% to 87.1%, and recall held roughly steady (85.3% $\rightarrow$ 82.2%), for an overall F1 improvement from 0.740 to 0.846. Consistent with a less relevance-biased model, the predicted-relevant rate across the full silver tier dropped from 75.4% to 61.2%. Finally, and most directly confirming the original diagnosis: 140 of 140 (100%) of the unanimously-judged spot-check candidates this time fell under a query the classifier had been trained on, compared to only 8 of 132 (6%) previously. The out-of-distribution generalisation gap identified in the first spot-check was not a flaw in the distillation approach itself, it was a direct consequence of training on 5 of 101 query topics, and expanding query coverage resolved it.

A further possible extension (not yet implemented). A complementary, cheaper way to grow labelled data further, without spending additional judging budget at all, would be to propagate labels from already-judged candidates to their nearest neighbours in embedding space: since every candidate already has multiple similarity scores computed against its query (Section 4.21), an unjudged candidate that is highly similar to a known highly-relevant gold-labelled company for the same query could plausibly inherit a provisional label, following the classic label propagation approach of [Zhu and Ghahramani \(2002\)](#). This is distinct from the distillation classifier already in use here, which learns a general relevance function from retrieval features, rather than propagating labels from specific labelled neighbours, and the two could in principle be combined or compared. This is left as a direction for future work rather than implemented in this thesis.

4.23.2 Deepening a Headline Query Subset via Round-Based Active Learning

The shallow, roughly uniform depth left by Section 4.23.1’s expansion (10-14 gold-labelled candidates per query) fixes out-of-distribution generalisation but is too thin to report reliable per-query Recall@ k /NDCG@ k as an evaluation benchmark, deeper judged pools reduce the variance of such metrics (Section 4.24 discusses this further). Rather than deepening all 101 queries, a set of 14 queries was selected for methodological diversity beyond simple topic variety: three testing geography not covered by the original pilot, one directly probing the BM25 vocabulary-mismatch failure mode (“venture capital firms”), four forming two adjacent-category disambiguation pairs (pharmaceuti-

4 Experiments

cal/biotech, investment/wealth management), two acronym-heavy queries, two niche or emerging industries, and two further sector-diversity queries.

Unlike the one-shot batch in Section 4.23.1, this used genuinely iterative active learning: label a round, retrain the classifier on everything labelled so far, re-rank the *remaining* candidates with the *updated* classifier, and repeat, since uncertainty estimates go stale as soon as new labels are added. A small fixed validation set (100 candidates, judged once with the stronger judge pair and reused for free across every round) tracked whether each additional round was still worth its cost.

Table 4.18: Round-based active learning trajectory for the 14 headline queries: cumulative new gold labels and classifier accuracy against the fixed validation set after each round.

Round	New labels this round	Cumulative labels	Validation accuracy
1	280	280	79%
2	280	560	83% (+4pp)
3	280	840	83% (+0pp)

Stopping. Round 3 produced no further improvement over round 2, a clean plateau after two rounds of genuine gain, which is exactly the stopping signal the method was designed to detect (Section 4.23.2’s round-based design follows the general active-learning stopping-criterion principle of monitoring diminishing returns (Settles 2009)). Further rounds were not run. One caveat: the validation set is only 100 candidates, so its standard error on a proportion near 80% is itself roughly 4 percentage points, meaning the round-2 gain sits close to the edge of what is statistically distinguishable from noise on its own; the plateau at round 3 (an exact repeat, not a smaller but still-positive gain) is the more convincing part of the signal.

Result. The 14 headline queries now hold 840 newly gold-labelled candidates on top of the 140 already present (10 per query from Section 4.23.1), roughly **70 gold-labelled candidates per headline query**, short of the 100-300 per query considered ideal for a fully reliable per-query evaluation benchmark (Section 4.24), but a deliberate, cost-justified depth rather than an arbitrarily chosen one.

Refreshing the silver tier at no additional cost. The combined 2,269-candidate gold standard (1,429 from Section 4.23.1 plus 840 from this section) was used to retrain the distillation classifier from Section 4.23 once more, and applied to the existing candidate pool to produce up to 1,000 silver-labelled candidates per query, capped rather than kept at the full pool size, and with every gold-verified candidate guaranteed a place regardless of its predicted probability rank. This required no new retrieval, encoding, or judging: every one of the 101 queries already has at least 1,022 candidates with retrieval features computed (median 1,661), so the only new work is classifier inference. All

101 queries now have exactly 1,000 candidates each (101,000 rows total, 2,267 of them gold-verified), the deepest per-query coverage produced so far in this thesis, though still silver-tier and bounded by the classifier’s demonstrated accuracy (Section 4.23.2) rather than independently verified.

4.23.3 Quantifying How Far Production’s Own Ranking Diverges from the Independent Gold/Silver Set

`dataset/production_results.xlsx`, production’s own top-1,000-per-query ranking, has exactly the same shape as the refreshed silver set above (101,000 rows, 1,000 per query, identical `query_id/domain` keys), making a direct, row-level comparison possible with no new judging or encoding cost. Four checks quantify what has, until this point, mostly been argued qualitatively in this thesis: how much production’s own output actually diverges from an independently verified standard.

Table 4.19: Four checks comparing production’s own top-1,000-per-query ranking against the independent gold/silver set.

Check	Result
1. Overlap	Mean 77.2% overlap between production’s top-1,000 and the independent top-1,000 per query (median 81.0%), but with real spread: query 1 (“software companies”, the query used throughout Section 4.21’s qualitative BM25 discussion) overlaps only 34.9%.
2. Are production’s own picks accurate?	Of production’s top-1,000 picks that happen to already have an independent gold label (1,434 candidates), 76.0% are confirmed highly relevant, 22.2% only partially relevant, and 1.8% not relevant at all.
3. Coverage gap	Of 1,499 gold-verified highly relevant candidates, 27.3% (409) are entirely absent from production’s own top-1,000 ranking , not ranked lower, simply never surfaced. Worst-affected queries: “wealth management firms” (45 missed), “investment management firms” (40), “software companies” (27).
4. Rank agreement	For the 77,993 candidates present in both rankings (spanning all 101 queries), the mean per-query Spearman correlation between production’s rank and the independently-trained classifier’s predicted probability is -0.056 , essentially no relationship, far from the strongly negative correlation (rank and predicted relevance moving in opposite, i.e. agreeing, directions) that would indicate the two rankings agree.

Finding. These four checks quantify, rather than merely argue, the circularity concern raised at the start of this thesis (Chapter 1): production’s own ranking is right more often than not when judged against the independent gold standard (Check 2), but it misses over a quarter of genuinely relevant companies outright (Check 3), and even among candidates both systems agree are worth considering, production’s internal ranking carries almost no relationship to independently-verified relevance (Check 4, -0.056). Benchmarking this thesis’s retrieval pipeline against production as a strong baseline (as recommended by Istari’s own engineering team, Section 1) remains the right framing, but these results make explicit exactly how much headroom that baseline leaves, both in what it fails to surface at all and in how loosely its internal ranking tracks true relevance among what it does surface.

4.23.4 A Direct Ranking Comparison: Production vs. an Independently Cross-Validated Ranker

Section 4.23.3 shows production’s ranking *differs* from the independent gold/silver set; this section asks whether it is actually *worse* at finding truly relevant companies, a stronger and more specific claim. Production’s own top-1,000-per-query ranking is compared against a ranking produced by the fusion-ranker classifier trained with leave-one-query-out cross-validation (the same principle as Section 4.22.2, extended from 5 to all 101 queries: for every query, the classifier is trained on every *other* query’s gold labels only, so no ranking is ever produced by a model that saw that query’s own labels). All 101 folds train in under 15 seconds, so this adds no meaningful cost. Both rankings are scored against the same gold-verified relevant set (`gold_label = 2`) using $\text{Recall}@k/\text{Precision}@k/\text{NDCG}@k$ at $k \in \{10, 50, 100, 300, 500, 1000\}$.

Table 4.20: $\text{Recall}@k$: production’s own ranking vs. the independently cross-validated ranking, against the same gold-verified relevant set.

System	Recall@ k					
	10	50	100	300	500	1000
<i>All 101 queries</i>						
Production (GOI)	0.011	0.048	0.088	0.225	0.366	0.720
Ours (cross-validated)	0.002	0.012	0.018	0.066	0.180	0.659
<i>19 deeply-labelled queries only (5 pilot + 14 headline, Sections 4.22.1, 4.23.2)</i>						
Production (GOI)	0.050	0.118	0.165	0.332	0.459	0.746
Ours (cross-validated)	0.012	0.042	0.073	0.183	0.325	0.654

Finding, stated honestly. Production’s own ranking outperforms the independently cross-validated ranking at *every* k value tested, on both the full 101-query set and the 19 deeply-labelled queries. This is not the result this thesis’s other findings would predict, and it is reported here without qualification rather than omitted. The most plausible

explanation lies in the composition of the gold-standard candidate pool itself, not in the correctness of the labels: the pooling methodology (Section 4.22) drew candidates from six sources, one of which was production’s own frozen output, and Section 4.23.3’s Check 2 already showed that 72.7% of gold-verified relevant companies were already present in production’s own top-1,000. Production therefore receives a structural advantage on this specific comparison: a large share of what later became a confirmed-relevant label was, by construction, a company production itself had already surfaced. This does not undermine the label-correctness evidence elsewhere in this thesis (the labels themselves are independently judged, Section 4.22.1), but it does mean this particular head-to-head ranking comparison cannot yet support a claim that this thesis’s ranking algorithm retrieves relevant companies more effectively than production does. Testing that claim fairly requires a gold-verified relevant set drawn from a candidate pool production had no part in constructing, which is exactly what the random 300,000-company sample (Section 4.24) is for; that comparison is left for future work pending the scaled encoding described there.

4.23.5 Is the Classifier’s Confidence Honest? A Calibration Check

The accuracy figures reported throughout Sections 4.23–4.23.2 measure whether the classifier’s predicted label (relevant/not, at the 0.5 cutoff) is correct. A separate question is whether its predicted *probability* is honest, when it reports 90% confidence, is it right about 90% of the time. This matters because the overwhelming majority of the silver tier was never individually spot-checked; trusting the classifier’s confidence at face value depends on it being well calibrated, not just accurate at one threshold. Using the same leave-one-query-out cross-validated predictions as Section 4.23.4, predicted probabilities for all 2,267 gold-labelled candidates were binned into deciles and compared against the actual relevant rate within each bin.

Table 4.21: Calibration: mean predicted probability vs. actual relevant rate, per decile (cross-validated).

Predicted probability bin	n	Mean predicted	Actual relevant rate
0.0–0.1	50	0.070	0.220
0.1–0.2	161	0.149	0.360
0.2–0.3	230	0.253	0.539
0.3–0.4	314	0.352	0.589
0.4–0.5	266	0.446	0.545
0.5–0.6	203	0.553	0.675
0.6–0.7	212	0.647	0.708
0.7–0.8	187	0.750	0.695
0.8–0.9	238	0.856	0.815
0.9–1.0	406	0.960	0.899

Finding. The overall Brier score is 0.220 (0 = perfect, 0.25 = no better than always predicting 0.5), a real but modest improvement over chance, consistent with the modest classifier accuracy reported elsewhere. The calibration curve is asymmetric: in the high-confidence range (0.7–1.0) the classifier is close to well-calibrated, only mildly overconfident. In the low-to-middle range (0.2–0.5), it is substantially *underconfident*: candidates the model scores around 0.25 turn out relevant 53.9% of the time, more than double the model’s own stated probability. Practically, this means the simple 0.5 cutoff used throughout this thesis to define “predicted relevant” is likely under-counting true positives that happen to fall in this band, offering a plausible partial explanation for part of the coverage gap documented in Section 4.23.3, and a concrete direction for future work (probability recalibration, e.g. Platt scaling or isotonic regression, before applying a hard threshold).

4.23.6 Honest Synthesis: What Has This Thesis’s Validation and Experimentation Actually Shown?

Part 1: is the gold/silver standard better than GOI’s own output? Pulling together every check in Sections 4.23–4.23.5, the answer is not a single yes or no, it depends on which specific claim is being made, and the evidence does not point the same way for each:

- **Is the labelling methodology more defensible than trusting production’s own output as ground truth?** Yes. Production’s ranking cannot validate itself; it is circular by construction (Chapter 1). This thesis’s gold standard uses independent multi-judge LLM labelling, blind calibration (Section 4.22.1), and human tie-breaking, none of which production’s own ranking has any equivalent of.
- **Does this thesis’s methodology find relevant companies production’s own system misses entirely?** Yes, demonstrated: 27.3% of gold-verified highly relevant companies are entirely absent from production’s top-1,000 (Section 4.23.3, Check 3).
- **Does this thesis’s ranking algorithm outperform production’s ranking at retrieving relevant companies, head-to-head?** Not yet shown, and the one direct test conducted (Section 4.23.4) found the opposite, likely attributable to residual candidate-pool bias rather than a genuine quality reversal, but this has not been independently confirmed.
- **Is the classifier itself highly accurate and well-calibrated?** Only moderately. Brier score 0.220, with a real, systematic calibration gap in the low-to-middle confidence range (Section 4.23.5).

The defensible claim this thesis supports is therefore narrower than “the new labels are better”: *the independent gold standard is methodologically sounder for evaluation purposes and demonstrates concrete, measurable blind spots in production’s own ranking,*

but this thesis has not yet shown that its own ranking algorithm retrieves relevant companies more effectively than production does. Confirming or refuting that stronger claim requires evaluating against a gold-verified relevant set drawn from candidates production had no part in surfacing. The random, production-independent sample and nested scaling corpus described next (Section 4.24) were built with exactly this in mind, though the specific production-independent comparison itself remains future work (Chapter 5).

Part 2: once the gold standard existed, did tuning and fine-tuning the pipeline against it actually help? Sections 4.23.8–4.24.1 ran four further, genuinely independent experiments against that gold standard, each with its own leakage-free held-out comparison, and the results are again mixed rather than uniformly positive:

- **Does systematically tuning the fusion ranker’s hyperparameters help?** Only partially. Recall@1,000 and NDCG@1,000 improve modestly, but NDCG@10 falls by roughly 55%, a genuine trade-off between broad-list coverage and top-of-list precision, not an unambiguous win (Section 4.23.8).
- **Does a different model class, a small neural network, do better than the tuned tree ensemble?** Yes, and unlike every other tuning or fine-tuning experiment in this thesis, unambiguously: the neural network wins at every k value tested against both the untuned and tuned gradient-boosted tree ensemble (Section 4.23.9), the strongest and cleanest positive result of any improvement attempted here, though based on a single random architecture search rather than the fully cross-validated search used for the tree ensemble.
- **Does fine-tuning the pretrained reranker on this task’s own labels help?** It depends entirely on how the training signal is constructed. A first attempt, trained on severely imbalanced data (2.3% negative examples), made the reranker worse at every cutoff. A second attempt, with explicit hard negatives and a pairwise ranking loss, reversed this and genuinely beat the pretrained reranker at $k = 500$ and $k = 1,000$, while still trailing it at $k = 10$ and $k = 50$ (Section 4.23.10), the same top-of-list-versus-broad-list trade-off seen in the hyperparameter tuning result.
- **Does approximate search remain viable once the corpus is scaled up?** Yes, clearly, and this is the one experiment in this group without a trade-off: HNSW-based ANN search matches exact search’s Recall@1,000/NDCG@1,000 within 0.5–2.5% while running 25–78 $\times$ faster across every corpus tier tested, confirming at real (if still sub-20-million) scale the same favourable trade-off first observed on a synthetic corpus earlier in this thesis (Section 4.24.1).

A pattern recurs across three of these four experiments: whenever a model’s training or search objective is reweighted toward broad ranking quality (a large Recall@ k /NDCG@ k at high k), it tends to gain coverage at some cost to precision at the very top of the list, and neither direction is unconditionally better, the right choice depends on whether the

4 Experiments

pipeline’s downstream consumer looks at a handful of results directly or takes a broad candidate pool for further processing (in this pipeline’s own case, the latter, Section 4.19). The neural network result is the one clear exception to that pattern, and the ANN result is not subject to it at all, approximate search’s speed gain here costs essentially no quality anywhere in the ranking. Taken together with Part 1, this thesis’s honest position is that its methodology is demonstrably more rigorous than trusting production’s own output, that several concrete, measurable improvements to the pipeline itself have been found (a neural fusion ranker, a properly-trained fine-tuned reranker, and a large, safe efficiency gain from ANN search), and that the single strongest possible claim, that the resulting system retrieves relevant companies more effectively than production overall, remains open pending the production-independent evaluation described in the next section.

Limitation: precision is a lower bound, not an exact figure. Every precision@ k number reported in this thesis is computed against the pooled, actively-sampled gold standard, not an exhaustive judgment of every retrieved candidate. A retrieved company only counts as a true positive if it happens to have already been gold-labelled; a genuinely relevant company that was simply never pooled or judged contributes neither a true positive nor a false positive; it is silently invisible to the metric. Recall@ k is unaffected by this, since it is computed only over the set of candidates that do have a gold label. Precision@ k , however, is systematically biased downward by an unknown amount, since some fraction of a method’s retrieved-but-unlabelled candidates are almost certainly relevant and simply were never reviewed. This is a known consequence of evaluating against pooled rather than exhaustive judgments (Bar-Yossef and Gurevich 2008; Lu et al. 2016), not a flaw specific to this thesis’s pipeline, but it means precision figures throughout should be read as conservative lower bounds rather than exact values.

Future work: tightening this bound with more exhaustive LLM ranking. A more exhaustive per-query relevance pass, an LLM directly ranking or scoring every candidate in an already-pooled, multi-source candidate set (rather than the smaller, actively-sampled subset actually judged in this thesis), would shrink the gap between reported and true precision by leaving fewer retrieved candidates unjudged. This must still draw its candidates from a pooled, multi-source set rather than a single retrieval method’s own output, otherwise the same single-source circularity this thesis’s gold standard was built to avoid (Section 4.22.1) reappears one layer downstream, with the LLM only ever able to rank whatever one method chose to surface. Scaling this idea further, toward Istari’s full company database rather than the ~98 716- to-397 025-company samples used throughout this thesis, is a natural direction: the same pool-then-judge structure would need to repeat at that scale, with the resulting broader, automated relevance judgments used both as a larger gold standard and as the basis for the production top-1,000 retrieval this thesis’s methods ultimately aim to support. Neither extension is implemented here; both are left as concrete next steps.

4.23.7 Preparing an External Review: Where Existing Gold Coverage Actually Sits in the Ranking

Before sending ranked candidate lists to external reviewers at Istari (Kerem Cerit and Manav) for a joint relevance-label review, it is worth checking where the gold labels already collected sit within each query’s own ranked output, since that determines whether the top of the ranking can already be called trustworthy or whether it is still, in effect, unverified.

Two representative queries. Query 1 (“software companies”) and query 92 (“wealth management firms”) were chosen to send out, the former for its comparatively large existing label count, the latter for having the largest coverage gap identified in Section 4.23.3.

Method. For each query, the classifier-ranked top-1,000 candidates from Section 4.22.2’s silver-label pipeline were split into five rank bands (1–100, 101–300, 301–500, 501–700, 701–1,000), and every candidate already carrying a verified gold label was located within its band.

Table 4.22: Where existing gold-verified labels fall within each query’s own ranked list.

Rank band	<i>n</i> gold-verified	% highly relevant	% at least partial
<i>Query 1: software companies (99 gold-verified of 1,000)</i>			
1–100	9	100.0	100.0
101–300	12	100.0	100.0
301–500	9	100.0	100.0
501–700	11	100.0	100.0
701–1,000	58	55.2	100.0
<i>Query 92: wealth management firms (70 gold-verified of 1,000)</i>			
1–700	0	–	–
701–1,000	70	90.0	100.0

Finding. Gold coverage is not spread evenly across the ranking, it clusters heavily toward the bottom of each query’s top-1,000 list. For query 92, every single gold-verified candidate falls in ranks 701–1,000; ranks 1–700 have no independent verification at all. Query 1 has thin coverage (9–12 candidates per band) scattered through ranks 1–700, with the bulk (58 of 99) again sitting in ranks 701–1,000. This is a direct consequence of how the gold standard was built: active learning (Section 4.23.1) deliberately samples uncertain, boundary-case candidates for labelling, not the candidates the classifier is already most confident about, so verified labels naturally accumulate low in the ranking rather than high. Where gold labels do exist, relevance is generally high (90–100% for

4 Experiments

query 92, 100% in four of five bands for query 1) but not perfect, query 1’s rank 701–1,000 band drops to 55.2% highly relevant, so “the bottom of the ranking is still decent” is roughly supported but not guaranteed, and “the top 500 are the best of the best” cannot yet be confirmed internally: there is essentially no ground truth there to confirm it with.

Implication for the external review. This motivates exporting the *full* ranked top-1,000 list per query, in rank order, rather than a hand-picked top- N sample: rank order alone already front-loads exactly the part of the list with the least existing verification (ranks 1–700), so no additional stratified-sampling logic is needed. The export omits every internal signal (predicted probability, silver/gold tier, existing gold label) and reviewers are asked to label top-down for as long as time allows. Because already-gold-labelled candidates remain at their natural rank position, unflagged, agreement or disagreement between the external reviewers’ judgments and this thesis’s own gold labels on those specific rows serves as a free, blind inter-rater consistency check once the review comes back.

4.23.8 Hyperparameter Tuning the Fusion Ranker

Every fusion ranker result reported so far, the original model (Section 4.21), the pooled-gold-standard retraining (Section 4.22.2), and the cross-validated ranking comparison against production (Section 4.23.4), uses the same hand-picked `HistGradientBoostingClassifier` configuration (`max_iter=150`, `max_depth=4`, `class_weight="balanced"`), chosen once early on and never systematically tuned against the gold standard. This section addresses that gap with a randomized hyperparameter search.

Method. The search space covers `max_depth` (2 to 8, or unlimited), `max_iter` (50 to 300), `learning_rate` (0.03 to 0.3), `min_samples_leaf` (5 to 30), and `l2_regularization` (0.0 to 2.0); `class_weight="balanced"` is kept fixed throughout, since it is an established, justified design choice rather than a model-capacity hyperparameter. 40 combinations are sampled at random (fixed seed for reproducibility).

Avoiding leakage in model selection. Selecting the best combination using the same cross-validated metric later reported as the final result would bias that result upward (the “winner’s curse” in model selection): a combination is not a fair, unbiased estimate of unseen-data performance if it was chosen *because* it scored best on exactly the data used to report it. To avoid this, the search itself is restricted to the 82 queries without deep gold coverage, evaluated with `LeaveOneGroupOut` cross-validation grouped by `query_id` within that 82-query pool only, scored by mean Recall@1,000 and NDCG@1,000. The 19 deep-coverage queries (5-query pilot plus 14 headline queries) are never touched during the search. Once the best combination is fixed, it is evaluated fresh with a full `LeaveOneGroupOut` pass over all 101 queries, the same protocol used for the untuned baseline (Section 4.23.4). The 19-deep-query subset of that final evaluation is a genuinely

4 Experiments

leakage-free comparison against the untuned baseline, since those queries never influenced which hyperparameters were chosen; the all-101-query subset is reported alongside it for comparability with earlier sections, but should be read as directional rather than strictly unbiased, since the other 82 queries did inform the search.

Results. The best combination found by the search (selected on the 82 tuning-only queries) was `max_depth=3`, `max_iter=50`, `learning_rate=0.03`, `min_samples_leaf=10`, `l2_regularization=0.5`, notably shallower and using far fewer boosting iterations than the original hand-picked configuration (`max_depth=4`, `max_iter=150`).

Table 4.23: Recall@ k and NDCG@ k on the 19 deep-coverage queries (leakage-free: these queries never contributed to the hyperparameter search), untuned baseline vs. the best combination found.

Model	Recall@ k					
	10	50	100	300	500	1000
Baseline (untuned)	0.012	0.042	0.073	0.183	0.325	0.654
Tuned	0.010	0.051	0.070	0.206	0.360	0.678

Model	NDCG@ k					
	10	50	100	300	500	1000
Baseline (untuned)	0.034	0.039	0.049	0.101	0.156	0.286
Tuned	0.015	0.033	0.044	0.108	0.168	0.295

Finding, stated honestly. Tuning produces a modest, genuine improvement on the metric the search was told to optimize: Recall@1,000 rises from 0.654 to 0.678 and NDCG@1,000 from 0.286 to 0.295, both leakage-free numbers since the 19 deep queries never influenced the search. This is not, however, an unambiguous win. NDCG@10 falls from 0.034 to 0.015, roughly a 55% relative decline, and the same pattern holds throughout the k -sweep: the tuned model beats the baseline at $k \in \{300, 500, 1,000\}$ but loses at $k \in \{10, 50, 100\}$. This is a direct, mechanistic consequence of the search objective, NDCG@1,000 and Recall@1,000 reward broad coverage of the full ranked list, not sharpness at the very top, so the search found hyperparameters (shallower trees, fewer boosting iterations) that spread relevance signal more evenly across the ranking at a real cost to top-of-list precision. Whether the tuned or untuned configuration is actually preferable depends on how the ranking is consumed downstream: the untuned baseline is better suited to a use case that only looks at the first handful of results, while the tuned configuration is closer to how this pipeline is actually deployed, a broad top-1,000 candidate pool handed to a downstream reranker (Section 4.19), which specifically benefits from higher recall further down the list rather than from top-10 precision on its own. On the all-101-query subset (not leakage-free, since 82 of those queries informed the search), the same qualitative pattern holds: NDCG@1,000 improves (0.166 $\rightarrow$ 0.173)

while NDCG@10 degrades ($0.007 \rightarrow 0.003$), reported for completeness rather than as independent confirmation.

4.23.9 A Neural Network Fusion Ranker

Every fusion-ranker result so far, tuned or not, uses `HistGradientBoostingClassifier`, a tree ensemble. This section tries a genuinely different model class on the same ten features: a small multi-layer perceptron (MLP), with its own hyperparameter search over architecture (hidden layer sizes and depth), optimization (learning rate), and regularization (dropout, weight decay), asking whether a neural network does any better than the tuned tree ensemble from Section 4.23.8.

Method. Unlike the GBDT search, full `LeaveOneGroupOut` cross-validation per hyperparameter combination is too expensive here, each fold trains a network from scratch rather than fitting a near-instant tree, so the search phase uses a single, fixed 80/20 split of the 82 tuning-only queries (66 train, 16 validation), never touching the 19 held-out deep queries; this is a standard, well-justified simplification for neural hyperparameter search. 20 combinations are sampled at random over hidden layer sizes ($\{16\}$ to $\{128, 64, 32\}$), dropout (0 to 0.5), learning rate (10^{-4} to 5×10^{-3}), and weight decay (0 to 10^{-2}), each scored by NDCG@1,000 on the 16 validation queries. Two details specific to neural networks that a tree ensemble does not need: features are standardized (zero mean, unit variance, scaler fit on the training split only) before being fed to the network, since gradient descent, unlike a tree split, is sensitive to feature scale; and the loss function is a class-weighted binary cross-entropy (weight inversely proportional to class frequency), mirroring the `class_weight="balanced"` convention used by every other classifier in this thesis. Once the best combination is fixed, it is evaluated with the same full `LeaveOneGroupOut` protocol over all 101 queries used everywhere else, so the comparison against the tuned GBDT on the 19 leakage-free deep queries is direct. Bayesian optimization (mentioned as a candidate search strategy alongside grid and random search) was not implemented, it would require adding `optuna` as a new dependency, and random search already gave a workable answer at negligible marginal cost given the searches already run in this thesis; noted here as a direction rather than silently dropped.

Results. The best architecture found, selected purely on the 16 search-validation queries, was notably small: a single hidden layer of 16 units, dropout 0.1, learning rate 10^{-4} , weight decay 10^{-3} .

Finding, stated honestly. Unlike every other tuning or fine-tuning experiment in this thesis, all of which traded top-of-list precision for broad-list recall or vice versa, the neural network wins at *every single* k value tested, against both the untuned and the tuned GBDT, on Recall@ k and NDCG@ k alike (Table 4.24; the same pattern holds on Recall@ k , not shown here for space). The gain is largest at the top of the list: NDCG@10 nearly triples over the untuned baseline ($0.034 \rightarrow 0.086$) and roughly sextuples over

4 Experiments

Table 4.24: NDCG@ k on the 19 held-out deep-coverage queries (leakage-free for all three models): untuned GBDT, tuned GBDT (Section 4.23.8), and the neural network.

Model	10	50	100	300	500	1000
GBDT baseline (untuned)	0.034	0.039	0.049	0.101	0.156	0.286
GBDT tuned	0.015	0.033	0.044	0.108	0.168	0.295
Neural network	0.086	0.079	0.094	0.165	0.222	0.360

the tuned GBDT ($0.015 \rightarrow 0.086$), while NDCG@1,000 improves by a smaller but still substantial margin ($0.286 \rightarrow 0.360$).

The most plausible explanation lies in what the winning architecture actually is: a single hidden layer of 16 units is a small, close-to-linear model, not a deep network exploiting complex feature interactions. This thesis’s ten fusion-ranker features are all continuous and already fairly informative (similarity scores, inverse ranks, the pretrained reranker’s own score), the kind of smooth, well-behaved feature space where a compact network with smooth decision boundaries can plausibly outperform an ensemble of axis-aligned tree splits, particularly once combined with standardized features and class-weighted loss, both necessary for a neural network but irrelevant to a tree-based model. This result should still be read cautiously: it comes from a single random architecture search over 20 combinations and a single train/validation split rather than the full cross-validated search used for the GBDT, so the specific margin should not be over-interpreted, but the direction, a small, properly regularized neural network outperforming the tuned tree ensemble at every cutoff on a leakage-free held-out set, is a genuine, unambiguous finding, the strongest and cleanest result of any tuning or fine-tuning experiment attempted in this thesis.

4.23.10 Fine-Tuning the Reranker

Every use of BAAI/bge-reranker-v2-m3 throughout this thesis, in the hybrid reranking pipeline (Section 4.19) and as one of the ten fusion-ranker features, treats it as a frozen, pretrained component, never fit to this thesis’s own queries or labels. This section fine-tunes it directly on the gold-labelled (query, candidate) pairs and checks whether that improves ranking quality, reusing the same 82-tuning/19-held-out query split as Section 4.23.8 rather than introducing a new one, so the two experiments stay directly comparable and the held-out queries remain untouched by either.

Method. The reranker is fine-tuned as a regression problem: for each gold-labelled (query, candidate-summary) pair from the 82 tuning-only queries ($\sim 1,000$ examples), the target score is `gold_label/2` (mapping 0/1/2 to 0.0/0.5/1.0, matching the reranker’s own continuous score output), trained with MSE loss, AdamW, learning rate 2×10^{-5} , batch size 32, for 4 epochs. This is a small dataset for a ~ 568 M-parameter model, a real overfitting risk flagged explicitly rather than glossed over. Both the original pretrained

4 Experiments

reranker and the fine-tuned version are then evaluated on the same task the reranker actually performs in this pipeline: reranking each held-out query’s full candidate pool from the union-pool feature set (Section 4.21, $\sim 1,000$ -2,900 candidates per query) and scoring the result by $\text{Recall}@k/\text{NDCG}@k$ against the gold-verified relevant set, restricted to the 19 deep-coverage queries so the comparison is leakage-free by the same logic as Section 4.23.8.

Table 4.25: $\text{Recall}@k$ and $\text{NDCG}@k$ on the 19 held-out deep-coverage queries: pretrained `bge-reranker-v2-m3` vs. the fine-tuned version.

Model	Recall@ k					
	10	50	100	300	500	1000
Pretrained	0.044	0.100	0.134	0.276	0.365	0.681
Fine-tuned	0.027	0.089	0.132	0.276	0.363	0.680

Model	NDCG@ k					
	10	50	100	300	500	1000
Pretrained	0.137	0.110	0.115	0.187	0.227	0.350
Fine-tuned	0.078	0.087	0.098	0.170	0.209	0.334

Finding, stated honestly. Fine-tuning made the reranker worse, not better, at every single k value tested, on precision, recall, and NDCG alike (Table 4.25). This is a clean negative result, not a marginal or mixed one: unlike Section 4.23.8’s trade-off between top-of-list and broad-list quality, there is no k at which the fine-tuned model wins.

The most plausible cause is visible directly in the training data: of the 820 training examples, only 19 (2.3%) were labelled `gold_label = 0` (not relevant); 594 (72.4%) were labelled highly relevant. Training loss fell sharply within two epochs ($0.646 \rightarrow 0.081$), consistent with the $\sim 568\text{M}$ -parameter model rapidly fitting a training signal that is overwhelmingly “this is relevant”, at the expense of the discriminative ranking ability the pretrained checkpoint already had. A reranker that has partly collapsed toward scoring most candidates as relevant would produce exactly this pattern: uniformly worse ranking at every cutoff, since its ability to *distinguish* candidates, not just recognise relevance in isolation, is what degrades. This is consistent with, and reinforces, the general lesson from Section 4.23.8: what a model is trained or tuned against determines what it gets good at, and a training signal this skewed was never likely to preserve, let alone improve, ranking quality.

A second attempt: hard negatives and a pairwise ranking loss. Rather than abandoning fine-tuning, the diagnosed cause was addressed directly. Each training query’s negative pool was supplemented with candidates sampled from its own retrieval pool that were never gold-labelled at all (a standard, well-justified assumption given how sparse true relevance is in a raw candidate pool), giving 1,782 (positive, negative) train-

4 Experiments

ing pairs across 79 tuning queries, versus only 19 explicit negatives out of 820 individual examples before. Training switched from MSE regression on an absolute score to a pairwise margin ranking loss, directly optimizing for whether a relevant candidate outscores a non-relevant one for the same query, the thing Recall@ k /NDCG@ k actually measure, rather than an absolute score value.

Table 4.26: Recall@ k and NDCG@ k on the 19 held-out deep-coverage queries: pretrained reranker vs. the first (MSE, imbalanced) and second (pairwise, hard negatives) fine-tuning attempts.

Model	NDCG@ k					
	10	50	100	300	500	1000
Pretrained	0.137	0.110	0.115	0.187	0.227	0.350
Fine-tuned v1 (MSE)	0.078	0.087	0.098	0.170	0.209	0.334
Fine-tuned v2 (pairwise)	0.070	0.078	0.102	0.175	0.240	0.363

Finding, stated honestly. Fixing the class imbalance and switching to a pairwise loss produces a genuine improvement over the first attempt at nearly every k , and the fine-tuned model now actually *beats* the pretrained baseline at $k = 500$ (NDCG 0.240 vs. 0.227, Recall@500 0.435 vs. 0.365) and $k = 1,000$ (NDCG 0.363 vs. 0.350, Recall@1,000 0.744 vs. 0.681), the first time in this thesis’s fine-tuning experiments that a fine-tuned model outperforms its pretrained starting point. This is not, however, an unambiguous win: at $k = 10$ and $k = 50$ the fine-tuned model still trails the pretrained baseline. This is the same qualitative pattern already found in Section 4.23.8’s fusion-ranker tuning, a training signal built around broad ranking quality (here, distinguishing relevant from a large sampled pool of presumed-negative candidates) improves broad-list metrics at some cost to top-of-list precision. Taken together, both experiments point to the same conclusion: reweighting a fusion or reranking model’s training objective reliably trades top-of-list sharpness for broad-list coverage in this pipeline, and neither direction is unconditionally “better”, the right choice depends on whether the downstream use case looks at a handful of results or consumes a broad candidate pool for further processing.

4.24 Scaling to a Larger Corpus: a Random Sample and Nested Corpus Tiers

The 98 716-company corpus used throughout this thesis is not just query-gatekept (Chapter 1), it is also small relative to Istari’s real production dataset. The thesis title’s “at scale” claim needs a genuine scalability test, does the retrieval pipeline’s quality and latency hold up as the pool of companies to search grows substantially larger, not just a claim resting on the existing corpus size.

4 Experiments

A fully random sample. A collaborator at Istari (Kerem Cerit) provided a fully random, query-independent sample of 300 000 companies drawn from the underlying database, deliberately not filtered or weighted toward any of the 101 queries used elsewhere in this thesis. Randomness here matters methodologically: using this sample to check whether a retrieval system’s index or corpus omits genuinely relevant material, rather than to build more query-specific training data, follows the same logic as random-sampling approaches used to estimate search engine index coverage (Bar-Yossef and Gurevich 2008), applied here to check whether the retrieval pipeline still finds known-relevant companies once far more distractor companies are added, rather than to estimate absolute index coverage as in that work.

Data quality. The sample was profiled and lightly cleaned (data profiling script, not included as a numbered notebook section here for brevity). Of note: only 0.56% (1 691 companies) overlapped with the existing 98 716-company corpus, confirming the sample is genuinely independent rather than a re-draw of existing data. No duplicate domains, no empty summaries, and no literal-string placeholder values were found. State/district/municipality fields were missing 15.7%/60.4%/75.4% of the time respectively, expected sparsity given inconsistent administrative-division reporting across countries, not a data quality defect. A small fraction of rows (1.03%) shared an exact-duplicate summary with another row, flagged rather than dropped, since a shared summary does not necessarily indicate an invalid company record.

Nested corpus tiers. Rather than four independently drawn samples of increasing size, the scaling corpus is built as four *nested* tiers: 100 000, 200 000, 300 000, and $\sim 397\,000$ (all available) companies, each strictly containing the previous tier plus additional randomly drawn companies from one fixed shuffle. The existing 98 716-company corpus, which holds all gold-labelled data used for evaluation, is included in every tier without exception, so that a change in a retrieval metric between tiers can be attributed to the added distractor companies, not to gold-labelled companies being dropped from the search pool. This nested design keeps the four tiers directly comparable to one another, following the same logic that motivates careful control of pooling depth in retrieval evaluation more broadly (Lu et al. 2016).

Encoding. Every company in the combined pool ($\sim 397\,025$ companies) is encoded exactly once with each embedding model, rather than re-encoded separately per tier: an embedding is a fixed, corpus-independent property of a single company’s text, so encoding once and subsetting afterward is both correct and avoids redundant computation. BM25 is a deliberate exception, being a corpus-level statistical model rather than a per-document encoding (a term’s weight depends on its frequency across the whole indexed collection), its index must be rebuilt separately for each of the four tiers once the scaling evaluation itself is run. The pretrained reranker (`bge-reranker-v2-m3`) is not part of this encoding step at all, it scores (query, candidate) pairs at retrieval time only, applied to whichever candidates survive first-stage retrieval, regardless of corpus size.

Scope: beyond the three fusion-ranker features. The scaled encoding was not limited to the three models already inside the trained fusion ranker (MiniLM, GTE-large, Linq-Embed-Mistral). Every embedding model evaluated as a baseline earlier in this thesis (BGE-large, BGE-M3, Nomic, E5-Mistral, SFR-Mistral, SimLM) and ColBERTv2 was also encoded at the new scale. The motivation is the same circularity concern raised in Section 4.22.2: the original baseline comparison chapter, including the finding that BGE underperforms MiniLM, was evaluated entirely against the circular `goi-search_results.json` ground truth, before the independent, pooled gold standard existed, and has never been rechecked against it, unlike the fusion ranker’s own feature-importance finding, which was rechecked and found to partially reverse (Section 4.22.2). Re-scoring every baseline model’s already-computed top-1000 rankings against the current non-circular, 1,429-candidate gold standard, at zero additional API or GPU cost, is planned as a follow-up to check whether the reported model ranking (MiniLM best, BGE and BGE-M3 below it) survives independent verification the same way the fusion ranker’s finding was checked, consistent with the general principle that pooling depth and evaluation-set independence, not just raw retrieval scores, determine how trustworthy a reported ranking of methods actually is (Bar-Yossef and Gurevich 2008).

ColBERT is architecturally distinct from every other model here: it stores one embedding per token rather than one vector per company (`doc_embs.pt`, shape $[N, 192, 128]$, roughly 19.5 GB at this scale versus 4.85 GB for the original 98,716-company run), so it cannot simply be subset into corpus-size tiers the same way a single dense vector per company can; it is included nonetheless, for completeness against every method previously evaluated.

Status. The combined pool and nested tier membership have been constructed, and encoding is complete: all 10 models (MiniLM, GTE-large, BGE-large, BGE-M3, Nomic, Linq-Embed-Mistral, E5-Mistral, SFR-Mistral, SimLM, ColBERTv2) have produced a full embedding file covering all $\sim 397\,025$ companies. The original 98 716-company Linq-Embed-Mistral run took approximately 34 minutes of GPU time, spread across several resubmitted jobs due to this cluster’s per-job wall-time limit and lengthy model-loading time, at roughly $4\times$ the scale, encoding times for the large AutoModel-based encoders turned out shorter than that estimate once a Hugging Face Hub network-verification hang was diagnosed and fixed, typically well under 30 minutes of pure encoding once loading succeeds.

Two evaluation notebooks answer question (1), does $\text{Recall}@k/\text{NDCG}@k$ and query latency hold up as the corpus grows from roughly 100K to 400K companies, restricted to the 19 queries with the deepest and most reliable gold-standard coverage (the 5-query pilot plus the 14 headline queries from Section 4.23.1), so the resulting metrics are not read from an unreliably shallow label set. A per-tier BM25 index notebook builds a separate index for each of the four nested tiers (tokenizing the corpus once and reusing the tokenization across tiers, since only the IDF statistics change with tier size, not the tokens themselves). A scaling-evaluation notebook then covers BM25 alongside the three embedding models already inside the trained fusion ranker (MiniLM, GTE-large,

4 Experiments

Linq-Embed-Mistral), computing $\text{Recall}@k/\text{NDCG}@k$ against the gold standard and per-query retrieval latency at every tier, for every method (all using exact brute-force search, ANN is treated separately in Section 4.24.1). Every gold-relevant candidate for the 19 deep queries is present in all four tiers by construction, confirmed directly (906 of 906 relevant candidates found in even the smallest, 100K, tier), so any change in these metrics across tiers is attributable to the growing pool of distractor companies rather than relevant companies dropping out of the search pool.

Table 4.27: $\text{Recall}@1,000$ and $\text{NDCG}@1,000$ at the smallest and largest tiers, all four methods.

Method	Recall@1,000		NDCG@1,000	
	100K	400K	100K	400K
BM25	0.632	0.570	0.339	0.310
MiniLM	0.678	0.610	0.364	0.330
Linq-Embed-Mistral	0.762	0.632	0.371	0.308
GTE-large	0.750	0.638	0.410	0.361

Finding. Recall degrades gracefully for every method as the corpus grows $4\times$ (10–17% relative decline, not a cliff), consistent with more distractor companies competing for a fixed top-1,000 budget rather than any retrieval failure. Linq-Embed-Mistral leads at 100K but degrades faster than GTE-large as the corpus grows, so by 400K the two are close on recall while GTE-large holds a clear NDCG lead throughout, meaning GTE-large is not just finding relevant companies but ranking them better. Latency tells a separate story: at 400K companies, BM25 is the *slowest* of the four methods (647 ms/query), slower even than the 4,096-dimensional Linq-Embed-Mistral embeddings (455 ms), while GTE-large’s 1,024-dimensional dense vectors are by far the fastest (112 ms). GTE-large is therefore the strongest method overall at scale on both quality and speed, while BM25, despite its simplicity, is both the weakest on quality and the slowest at this corpus size. Question (2), whether the original baseline-model ranking survives re-evaluation against the non-circular gold standard, remains a separate follow-up not yet started.

4.24.1 Approximate Nearest Neighbor (ANN) Index Tuning

The thesis proposal commits explicitly to approximate nearest neighbor search (“supports scalable top- k retrieval using approximate nearest neighbor search”; “trade-off between retrieval accuracy and latency under different ANN index configurations”). Section 4.13 took a first pass at this question early in the thesis, using a synthetic 2-million-vector corpus (real embeddings padded with Gaussian-noise-perturbed filler vectors) and found a $6.5\text{--}6.6\times$ speedup with no meaningful quality loss. Every other result in this thesis, including the scaling evaluation above, still uses exact brute-force search on real data, a full dot product against every company in the tier. This section

4 Experiments

repeats the ANN question on the real, nested 100K–400K corpus built for the scaling evaluation, rather than a synthetic one, to check whether the earlier finding holds at genuine (if still sub-20-million) scale.

Method. A FAISS (Johnson et al. 2019) HNSW index is built over GTE-large’s embeddings (the strongest all-around performer from the scaling evaluation above) at each of the four corpus tiers. HNSW’s hyperparameters are index-structure settings rather than model-training settings: `M` (graph connections per node) and `ef_construction` (build-time search width) are fixed when an index is built; `ef_search` (query-time search width) can be swept cheaply against an already-built index without rebuilding. Two build configurations are tried per tier (`M=16/ef_construction=40`, a faster/lighter build, and `M=32/ef_construction=200`, a slower/higher-quality build), and `ef_search` is swept across {10, 50, 100, 200, 500, 1,000} for each. Unlike Section 4.23.8’s single best-combination search, there is no one “best” ANN setting, recall and latency trade off against each other by construction, so the natural output is a curve across settings, compared directly against the exact-search numbers in Table 4.27 for the same model, tiers, and queries.

Table 4.28: Exact search vs. ANN (HNSW, `M=16`, `ef_construction=40`, `ef_search=500`, the cheaper-to-build configuration) at each tier, GTE-large, Recall@1,000/NDCG@1,000, and query latency.

Tier	Recall@1,000		NDCG@1,000		Latency (ms)		Speedup
	Exact	ANN	Exact	ANN	Exact	ANN	
100K	0.750	0.735	0.410	0.405	28.3	1.13	25×
200K	0.702	0.684	0.390	0.383	57.0	1.27	45×
300K	0.663	0.656	0.373	0.371	85.1	1.36	62×
400K	0.638	0.635	0.361	0.359	112.3	1.44	78×

Finding. At a moderate `ef_search` setting (500), ANN retrieval matches exact search within 0.5–2.5% on both Recall@1,000 and NDCG@1,000 at every tier, while running 25–78× faster (Table 4.28). The speedup grows with corpus size, from 25× at 100K to 78× at 400K, since exact search’s cost scales with corpus size while HNSW’s query cost grows only logarithmically, so ANN’s advantage compounds exactly as the corpus scales further, directly supporting the proposal’s premise that approximate search is what makes scalable top-*k* retrieval practical. Higher-quality configurations can do marginally better still: `M=32/ef_construction=200` at `ef_search=200` reaches Recall@1,000=0.644 at the 400K tier, fractionally *above* exact search’s own 0.638 (an artefact of HNSW’s approximate graph traversal occasionally surfacing a marginally different top-1,000 set, not a meaningful quality gain), at 1.40 ms, still roughly 80× faster than exact search.

This is not a free lunch at every setting, the trade-off the proposal asks to characterize is real at the cheap end: at `ef_search=10`, recall collapses to 0.16–0.28 depending on tier

4 Experiments

and build configuration, far below exact search, confirming that ANN’s speed advantage is bought at real cost to recall once search width is set too aggressively. Build cost also differs sharply between configurations: at the 400K tier, `M=16/ef_construction=40` builds in 88.8 s versus 556.5 s for `M=32/ef_construction=200`, roughly $6\times$ slower, for a quality difference of at most 1–2% once `ef_search` is reasonably high. The cheaper build configuration is therefore the more practical choice overall: fast to build, fast to query, and recall/NDCG effectively indistinguishable from exact search once `ef_search` is set to 500 or above.

This experiment’s 25–78 $\times$ speedup is substantially larger than Section 4.13’s earlier 6.5–6.6 $\times$ finding on the synthetic 2-million-vector corpus; the two are not a strictly controlled comparison, since they use different embedding models, different exact-search implementations, and different corpora, so the specific magnitudes should not be compared directly. What matters is that both experiments, one on synthetic filler data early in this thesis, one on the real, nested scaling corpus built later, agree on the same qualitative conclusion: approximate search is a large, low-risk efficiency gain for this retrieval task, with negligible quality cost at a well-chosen operating point.

5 Conclusion and Future Work

5.1 Summary of Contributions

This thesis set out to answer a specific question (Chapter 1): can embedding-based semantic retrieval, combined with approximate nearest-neighbour search, generate accurate top- k candidate sets from a large company dataset while keeping query latency low, and how does it compare to Istari’s own production system? The evidence gathered across this thesis answers each part of that question directly rather than in the abstract.

On retrieval quality, a systematic comparison of more than ten dense embedding models, a late-interaction architecture, and several hybrid pipelines converged on a learned fusion ranker, trained on this task’s own queries rather than assembled from fixed, hand-designed rules, as the strongest method evaluated against the original pseudo-relevance protocol ($\text{Recall@1,000} = 0.823$, Section 4.21). Because no human relevance judgements exist for this task, that result alone would rest entirely on agreement with production’s own, potentially circular, output. This thesis therefore also built an independent, multi-judge, blind-calibrated gold standard from scratch (Section 4.22.1), expanded it via active learning to cover all 101 queries (Section 4.23.1), and used it to re-examine the fusion ranker’s own interpretive claims, revealing that its most prominent feature-importance finding did not survive independent validation (Section 4.22.2) and that production’s own ranking misses more than a quarter of independently verified relevant companies outright (Section 4.23.3). Further experiments tuned and fine-tuned the pipeline against this gold standard, and found genuine, if uneven, gains: a small neural network fusion ranker that outperformed the tuned tree ensemble at every cutoff (Section 4.23.9), and a properly-trained fine-tuned reranker that beat its pretrained starting point at deeper cutoffs once its training signal was corrected (Section 4.23.10).

On efficiency, the accuracy–efficiency trade-off central to this thesis’s research question was tested twice: first on a synthetic 2-million-vector corpus early on (Section 4.13), and again on a real, nested corpus of up to 397,025 companies built specifically to test scaling (Section 4.24). Both experiments reach the same conclusion: approximate nearest-neighbour search matches exact search’s retrieval quality within a few percentage points while running tens of times faster, and this advantage grows, not shrinks, as the corpus scales further (Section 4.24.1), directly supporting the efficiency half of this thesis’s research question.

Read honestly and without overclaiming (Section 4.23.6), this thesis demonstrates that its methodology is more rigorous than trusting a production system’s own output as ground truth, that concrete, measurable improvements to the retrieval pipeline exist and were found, and that approximate search keeps the resulting pipeline efficient at meaningfully larger scale than the original evaluation corpus. It does not yet

demonstrate, on a fair, production-independent test, that the resulting pipeline retrieves relevant companies more effectively than production does overall, a narrower and more specific claim than the individual pieces of evidence gathered here might suggest, and one this thesis is explicit about not having closed.

5.2 Future Work

Several concrete extensions were identified over the course of this thesis but not implemented, either for lack of time or because they depend on results not yet available. They are grouped here by what they would establish.

Closing the central open question. The single most important unresolved claim in this thesis is whether its retrieval pipeline outperforms production at finding relevant companies, tested fairly. The one direct test conducted (Section 4.23.4) used a gold standard partly pooled from production’s own output, giving production a structural advantage on that specific comparison. A fair test requires gold-verified relevant companies drawn from the random, production-independent 300,000-company sample already collected for the scaling experiments (Section 4.24), which production had no part in surfacing. Relatedly, re-scoring the original baseline embedding models’ already-computed rankings against the current, non-circular gold standard, at no additional retrieval or encoding cost, would check whether the original baseline comparison (Section 4.7) survives independent verification the same way the fusion ranker’s own feature-importance finding was checked and found to partially reverse (Section 4.22.2).

Strengthening the evaluation methodology itself. Precision figures throughout this thesis are conservative lower bounds rather than exact values, since the gold standard is pooled rather than exhaustive (Section 4.23.6); a more exhaustive LLM ranking pass over an already-pooled candidate set, rather than the smaller actively-sampled subset judged here, would tighten this bound. None of the many comparisons in this thesis (tuned vs. untuned, fine-tuned vs. pretrained, production vs. independent ranking) currently carry statistical significance tests across per-query metrics; adding these, cheap given the per-query results already computed, would make the strength of each finding more precise. The classifier’s predicted probabilities are only moderately well calibrated, with a systematic gap in the low-to-middle confidence range (Section 4.23.5); probability recalibration, for example Platt scaling or isotonic regression, before applying the fixed 0.5 relevance threshold used throughout is a direct, well-understood fix. Label propagation from judged candidates to their nearest, unjudged neighbours in embedding space (Section 4.23.1) remains a cheaper, complementary way to grow labelled data that was identified but not implemented. Finally, the joint relevance-label review with Istari’s engineering team (Section 4.23.7), sent out but not yet returned at the time of writing, would provide the first genuinely blind, non-circular human judgement on this thesis’s own gold labels, both validating them directly and testing whether the coverage gains claimed in Section 4.23.3 hold up under independent scrutiny.

Further modelling experiments. The neural network fusion ranker (Section 4.23.9) used a random hyperparameter search over a single train/validation split rather than the fully cross-validated search used for the tree ensemble; Bayesian optimisation, mentioned as a candidate strategy but not implemented for lack of a suitable dependency, could search the same space more efficiently and with a proper nested cross-validated estimate of its final performance. Fine-tuning was applied to the reranker (Section 4.23.10) but not to a bi-encoder embedding model directly, a larger and riskier undertaking given the modest size of the current gold standard, but one with real potential to improve first-stage retrieval quality rather than only reranking. The boilerplate/platform-summary contamination identified in the underlying corpus (Section 4.15) was quantified but not corrected in any result reported here; using the boilerplate-similarity score as an input feature to a trained model, rather than as a hard filter, would let the model learn how much to trust an imperfect signal instead of requiring it to be reliable enough for a fixed decision rule.

Scaling further. Every experiment in this thesis operates on a corpus of at most 397,025 companies, roughly 2% of Istari’s full 20-million-company database. The ANN scaling results (Sections 4.13, 4.24.1) are consistent across a synthetic 2-million-vector test and a real 397,025-company test, and both point toward the same favourable trade-off, but neither directly measures behaviour at the full production scale. Testing whether retrieval quality, ANN’s efficiency advantage, and the gold-standard construction pipeline itself all continue to hold at that scale would require a substantially larger data export from Istari than used in this thesis, and is the natural next step toward a production deployment of the methods developed here.

Bibliography

- BAAI. bge-reranker-v2-m3. <https://huggingface.co/BAAI/bge-reranker-v2-m3>, 2024. Model card.
- Ziv Bar-Yossef and Maxim Gurevich. Random sampling from a search engine’s index. *Journal of the ACM*, 55(5), 2008. doi: 10.1145/1411509.1411514.
- Steven Bird, Ewan Klein, and Edward Loper. NLTK: the natural language toolkit, 2009.
- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. ACM, 2020. doi: 10.1145/3397271.3401075.
- Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval*, 3(3):225–331, 2009. doi: 10.1561/15000000016.
- Xiaolu Lu, Alistair Moffat, and J. Shane Culpepper. The effect of pooling and evaluation depth on IR metrics. *Information Retrieval*, 2016. doi: 10.1007/s10791-016-9282-6.
- Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

Bibliography

- Mixedbread AI. mxbai-rerank-large-v1. <https://huggingface.co/mixedbread-ai/mxbai-rerank-large-v1>, 2024. Model card.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/15000000019.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272.
- Burr Settles. Active learning literature survey. Technical Report Computer Sciences Technical Report 1648, University of Wisconsin-Madison, 2009.
- Rikiya Takehi, Ellen M. Voorhees, Tetsuya Sakai, and Ian Soboroff. LLM-assisted relevance assessments: When should we ask LLMs for help?, 2024.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- Paul Thomas, Seth Spielman, Nick Craswell, and Bhaskar Mitra. Large language models can accurately predict searcher preferences, 2024.
- Ellen M Voorhees. The trec-8 question answering track report. In *Proceedings of TREC*, 1999.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Simlm: Pre-training with representation bottleneck for dense passage retrieval, 2022.
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 641–649. ACM, 2024. doi: 10.1145/3626772.3657878.

Bibliography

- Xiaojin Zhu and Zoubin Ghahramani. Learning from labeled and unlabeled data with label propagation. Technical Report CMU-CALD-02-107, Carnegie Mellon University, Pittsburgh, PA, 2002.
- Justin Zobel. How reliable are the results of large-scale information retrieval experiments? In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 307–314. ACM, 1998. doi: 10.1145/290941.291014.

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Tool	Purpose	Where?	Useful?

Unterschrift

Mannheim, den July 29, 2026